Fondamenti di Informatica - Modulo II

Giorgia Pazienza

1 Definizione Base

Che cos'è la Calcolabilità?
La Calcolabilità è la possibilità teorica di risolvere un problema automaticamente

Che cos'è la Complessità?
La Complessità è la possibilità effettiva/pratica di risolvere un problema

Che cos'è un Problema?
Un problema è la descrizione di un insieme di istanze, collegati da un certo insieme di relazioni. La descrizione è associata alla richiesta di derivare dai dati un altro insieme di paramenti che formano la soluzione.

Che cos'è un Istanza?
Un istanza del problema è un insieme di parametri, chiamati dati.

Che cos'è la Soluzione di un Istanza?
Una soluzione di un istanza è un insieme di valori, che rispettano l'insieme dei vincoli del problema.

Che cosa significa trovare la Soluzione di un Problema?
Significa estrarre informazione nascosta (implicita) dall'informazione in nostro possesso (esplicita).

Che cos'è un Algoritmo?
Un Algoritmo è la descrizione di una sequenza di passi elementari che permettono ad un qualche esecutore di calcolare una soluzione di problema a partire da una qualsiasi sua istanza.

Che cos'è un Passo Elementare?
Con questo terminale ci riferiamo ad un operazione il cui significato sia immediatamente comprensibile a chi dovrà eseguire effettivamente l'algoritmo. s

2 Introduciamo la Macchina di Turing

Problema: Un insieme di n contenitori numerati da 0 a n - 1, in ciascuno dei quali è contenuto un numero intero. Vogliamo scambiare i contenuti dei contenitori tale che a fine procedimento i numeri nei contenitori sono ordinati. (quindi disposti in maniera crescente).

Per risolvere questo problema usiamo l'algoritmo di *Bubble Sort*. L'algoritmo agisce nel seguente modo: per ogni coppia di contenitori consecutivi, vengono confrontati i loro contenuti e se non sono nell'ordine corretto vengono scambiati.

Alan Turing, studiando come le persone risolvono i problemi, si accorse che qualsiasi compito complesso può essere scomposto in una serie di passaggi semplicissimi. Ognuno di questi passaggi elementari (o operazioni) ha due caratteristiche chiave:

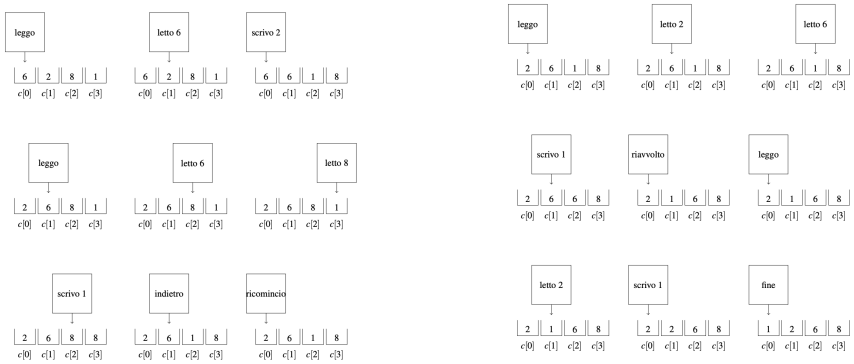
- Lavora solo su una parte molto limitata delle informazioni: L'operazione non deve guardare tutti i dati del problema contemporaneamente. Si concentra solo su un pezzetto di informazione alla volta (come leggere un singolo numero o un simbolo).
- Il risultato dipende da "cosa si sta facendo adesso": L'operazione da eseguire è sempre la stessa (ad esempio, "scrivi qui" o "spostati a destra"), ma l'azione specifica che viene eseguita e il suo effetto dipendono dallo stato attuale della persona o della macchina che la sta eseguendo.

Infatti Turing definì un modello di calcolo chiamato: Macchina di Turing
Immaginiamo di descrivere il funzionamento della Macchina di Turing pensando a come eseguirebbe il problema del *Bubble Sort* (definito precedentemente).
Inizialmente andrebbe ad esaminare il numero contenuto nel primo contenitore, poi ricordando tale numero andrebbe ad esaminare il numero contenuto nel secondo contenitore. Se tale numero fosse più piccolo di quello che è nella sua memoria allora:

- Consideriamo il numero più grande (Spostamento a destra): Scrive il numero sulla lavagna. Prende il numero più grande (quello che aveva in memoria, da M1) e lo scrive su M2, sovrascrivendo quello precedente. Contemporaneamente, memorizza il numero più piccolo (quello che era appena stato cancellato da M2).
- L'esecutore torna indietro al primo contenitore (M1) e scrive su di esso il numero più piccolo che aveva appena memorizzato. I due numeri sono stati scambiati. Adesso il numero più piccolo è in M1 e il numero più grande è in M2.
- Torna su M2, legge il numero più grande (che ora è lì) e lo memorizza. Ora si sposta al terzo contenitore (M3) per confrontare il numero che ha in memoria (il numero di M2) con quello che troverà in M3, ripetendo lo stesso processo di scambio se necessario.

Se il numero contenuto nel primo contenitore non fosse più grande di quello contenuto nel primo contenitore, allora l'esecutore memorizzerebbe il contenuto del secondo contenitore. Questo processo verrà ripetutto più volte, fino a giugnere all'ultimo contenitore.

Possiamo descrivere quindi la Macchina di Turing nel seguente modo:



- un dispositivo di calcolo dotato di una memoria di lavoro ad accesso sequenziale.
- in grado di leggere e scrivere dati su tale memoria
- ciò che viene scritto dipende da ciò che stava in memoria precedentemente.
- ha una memoria molto limitata.

Definizione Formale:
Sia Σ un alfabeto finito e Q un insieme finito di stati, con q_0 lo stato iniziale e Q_F l'insieme di stati finali. Una *Macchina di Turing* T sull'alfabeto Σ e sull'insieme di stati Q è un dispositivo di calcolo dotato di:

- un'unità di controllo, che ad ogni istante della computazione, può trovaarsi in un qualsiasi stato in Q
- di un nastro di lettura/scrittura, di lunghezza infinita, suddiviso in celle indicizzate, contenenti un carattere in Σ o un $\square$ (cella vuota)
- di una testina di lettura/scrittura che ad ogni istante della computazione è posizionata su una cella del nastro
- un programma P, ossia un insieme di quintiple $< q_1, a_1, a_2, q_2, m >$
  - q_1: Lo stato interno attuale della macchina
  - a_1: Il simbolo che la testina legge in memoria nell'attuale istante di tempo.

- a_2: Il nuovo simbolo che la testina deve scrivere in quella stessa cella.
- q_2: Lo stato in cui la macchina deve passare dopo aver eseguito l'azione
- m: La direzione in cui la testina deve spostarsi (sinistra, destra o ferma)

Che cos'è una computazione?
Una Computazione *T(x)* della macchina T su un input x è l'applicazione delle quintuple P ad x partendo dal primo simbolo di x con l'unità di controllo che si trova in q_0 di T.

N.B: L'esecuzione di una singola quintupla è un *passo elementare*.

3 Macchina di Turing a più Nastri

Le Macchine di Turing possono essere definite con un numero arbitrario di nastri. Su ogni nastro è presenta una testina di lettura/scrittura. I due nastri hanno molteplici funzionalità:

- è possibile inserire in un nastro pezzi di input diversi (prendendo come esempio il problema di prima, adattandolo in questo caso un nastro può contenere n e l'altro m)
- è possibile dedicare un nastro all'output e uno alla tringa di input
- è possibile utilizzare uno o più nastri come nastri di lavoro, ovvero per memorizzare i risultati intermedi dei calcoli e facilitare le operazioni della macchina.

Dividiamo in 2 categorie le macchine di Turing a più nastri:

- Macchine di Turing a *Testine Solidali*: Dopo l'esecuzione di una quintupla, le testine si muovono in modo sincronizzato, ovvero tutte eseguono lo stesso movimento (destra, sinistra, fermo). Le testine rimangono sempre posizionate sulla cella con lo stesso indirizzo su tutti i nastri (cioè al passo h, saranno tutte sui nastri in posizione 1). Una quintupla sarà formata nel seguente modo: $< q_i, s_1, s_2, q_j, mov >$ dove
  - $s_1 = (s_{1_1}, s_{1_2}, ..., s_{1_k}), s_2 = (s_{2_1}, s_{2_2}, ..., s_{2_k})$
  - mov sinistra,ferma,destra.
- Macchine di Turing a *Testine Indipendenti*: Dopo l'esecuzione di una quintupla, le testine si muovono in modo asincrono, ovvero tutte eseguono un movimento a se (destra, sinistra, fermo). Le testine rimangono posizionate su celle con diverso indirizzo su tutti i nastri (cioè al passo h, saranno tutte sui nastri in posizioni diverse). $<qi, s1, s2, qj, mov>$ dove
  - $s_1 = (s_{1_1}, s_{1_2}, ..., s_{1_k}), s_2 = (s_{2_1}, s_{2_2}, ..., s_{2_k})$
  - $mov = (mov_1, mov_2, ..., mov_k) mov_h \in sinistra, ferma, destra,$ per $h = 1, ..., k.$

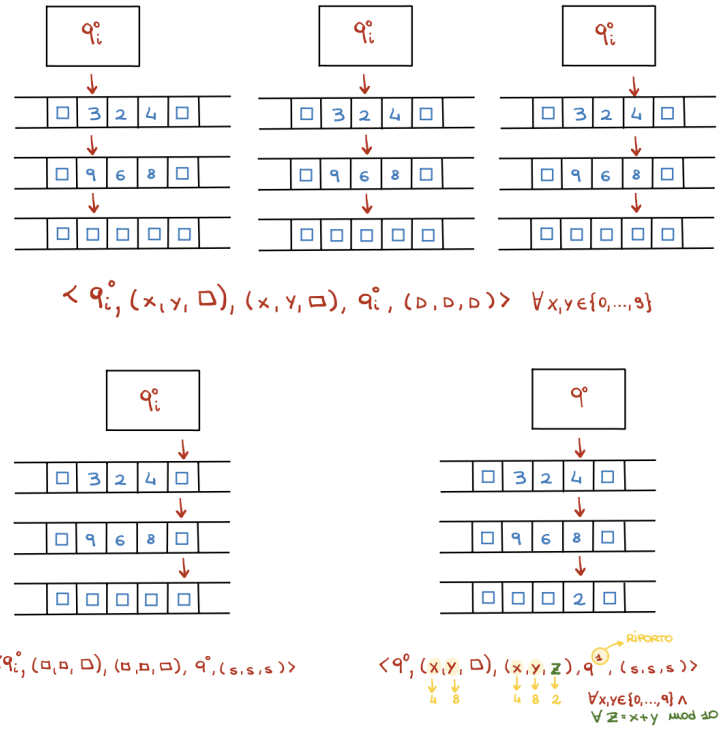
Esecizio 3.1: Risolviamo il problema della somma, con una macchina a 3 nastri:

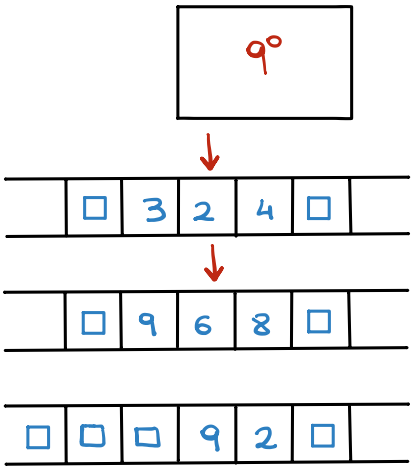
- 1 nastro per l'input n
- 1 nastro per l'input m
- 1 nastro per l'output n + m

Risolto con una macchina a 3 nastri a Testine Solidali

Definiamo la Macchina di Turing:

- L'alfabeto: $\Sigma = \{0, ..., 9; +\}$
- Lo stato Iniziale: q_i^0
- L'insieme degli Stati: $\{q_i^0, q^0, q^1, q_F\}$
- L'insieme degli stati Finali: $\{q_F\}$
- Le Quintuple: *Illustrate nelle pagine seguenti nell'esempio riportato*

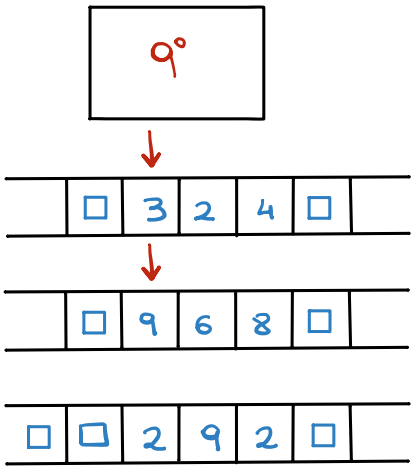




$\langle q^0, (x, y, \square), (x, y, z), q^0, (s, s, s) \rangle$

$x \to 2 \quad y \to 6 \quad x \to 2 \quad y \to 6 \quad z \to 9$

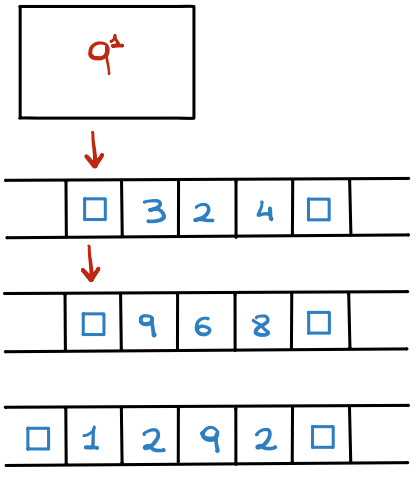
$\forall x, y \in \{0, \ldots, 9\} \wedge$
$\forall z = x + y \mod 10$



$\langle q^0, (x, y, \square), (x, y, z), q^1, (s, s, s) \rangle$

$x \to 3 \quad y \to 9 \quad x \to 3 \quad y \to 9 \quad z \to 2$

$\forall x, y \in \{0, \ldots, 9\} \wedge$
$\forall z = x + y \mod 10$



$\langle q^1, (\square, \square, \square), (\square, \square, z), q_F, (F, F, F) \rangle$

$z \to 1$

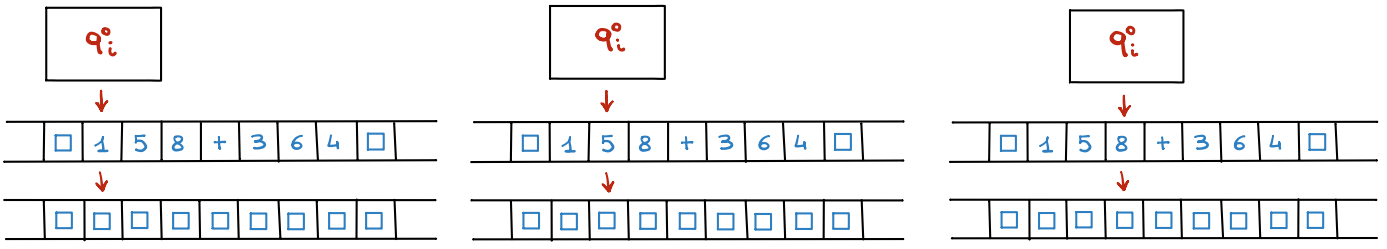
$\forall z = x + y \mod 10$

Esecizio 3.1: Risolviamo il problema della somma, come quello precedente, ma in linea. Useremo una macchina a 2 nastri:

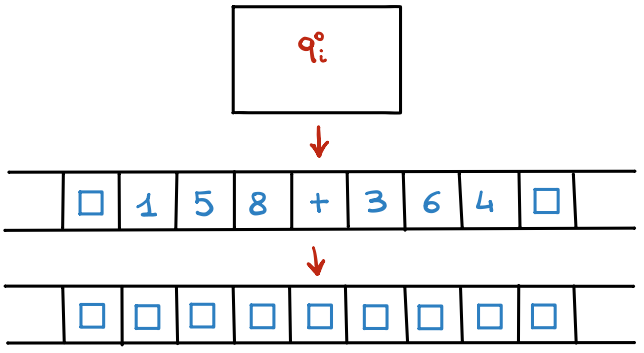
- 1 nastro per l'input
- 1 nastro per l'output

Definiamo la Macchina di Turing:

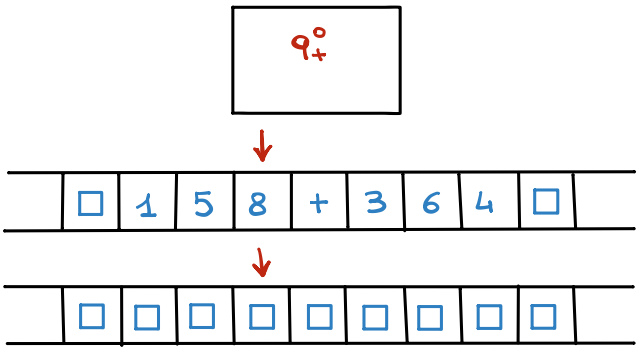
- L'alfabeto: $\Sigma = \{0, ..., 9; +\}$
- Lo stato Iniziale: q_i^0
- L'insieme degli Stati: $\{q_i^0, q_x^0, q_+, q^1, q_{ind}^1, q_{xind}, q_{+ind}^0, q_F\}$
- L'insieme degli stati Finali: $\{q_F\}$
- Le Quintuple: *Illustrate nelle pagine seguenti nell'esempio riportato*



$\langle q_i^\circ, (x,\square), (x,\square), q_i^\circ, (D,D)\rangle \quad x \in \{0,\dots,9\}$



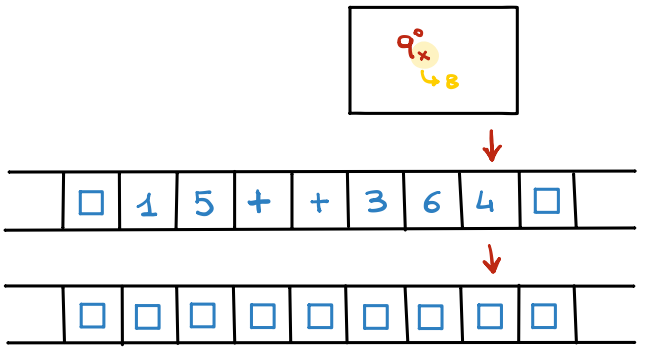
$\langle q_i^\circ, (+,\square), (+,\square), q_+^\circ, (S,S)\rangle$



$\langle q_+^\circ, (x,\square), (+,\square), q_x^\circ, (D,D)\rangle$

x: 8 — q_x: 8

$\forall x \in \{0,\dots,9\}$

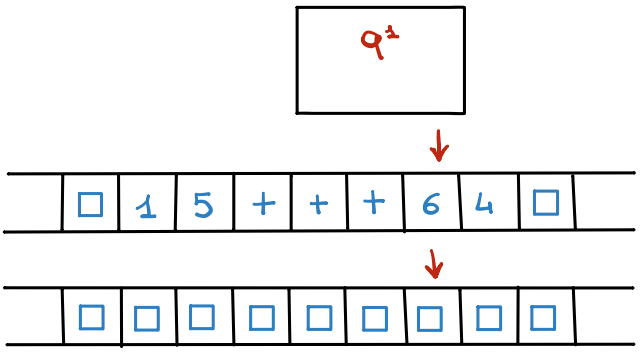


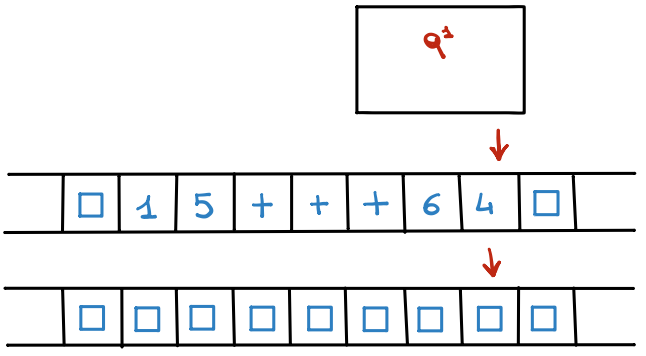
$\langle q_x^\circ, (x,\square), (+,z), q^{\text{RIPORTO}}, (D,D)\rangle$

x: 8 — x: 4 — z: 2

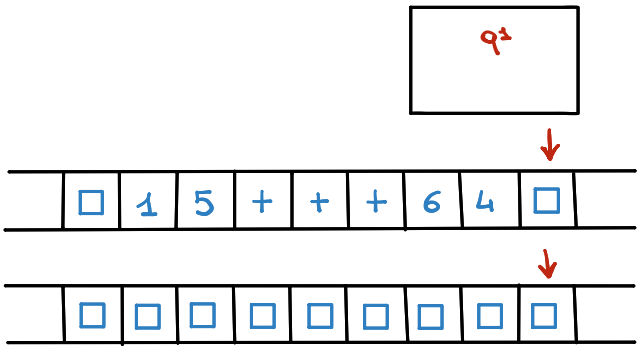
$\forall x \in \{0,\dots,9\}$

$z = x+y \bmod 10$

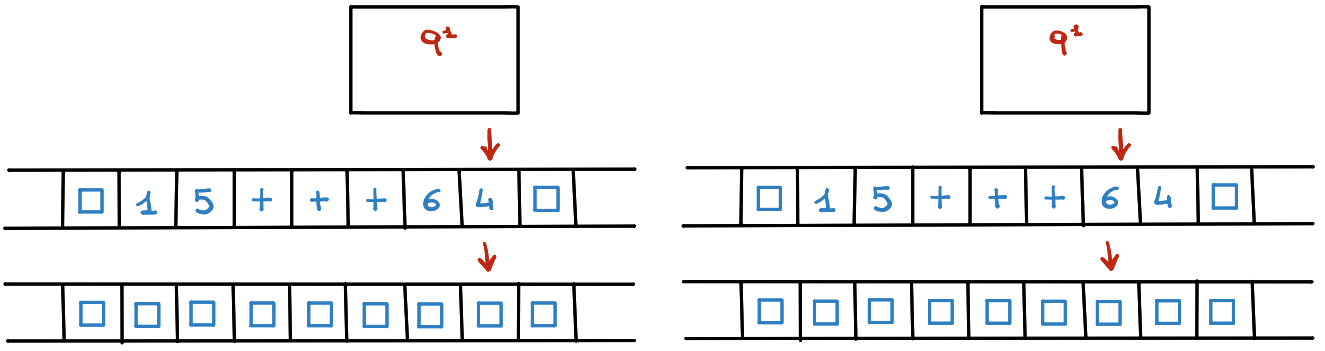




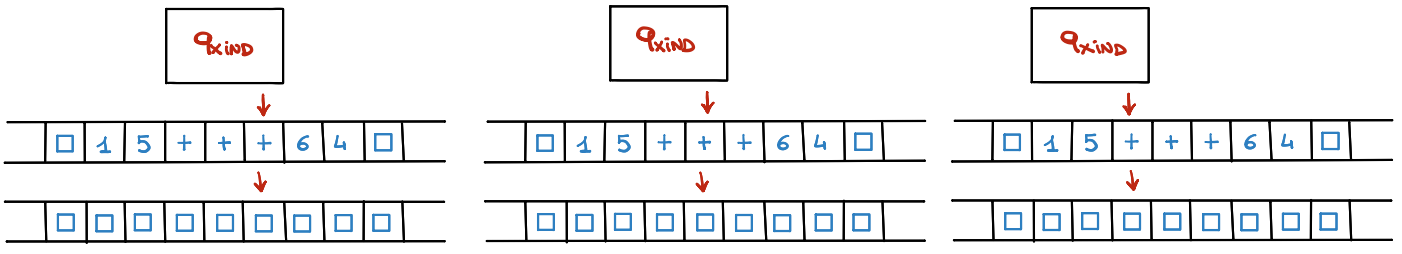
$\langle q^1, (x,\square), (x,\square), q^1, (D,D)\rangle$



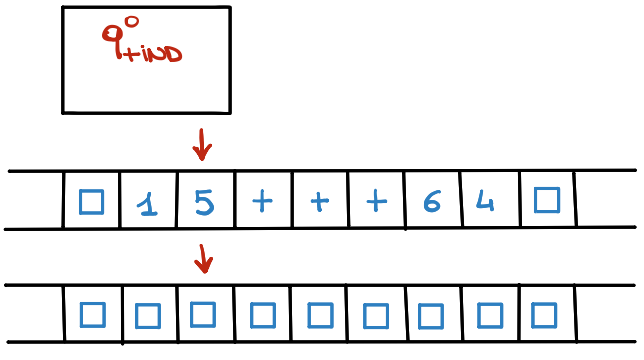
$\langle q^1, (\square,\square), (\square,\square), q^1_{ind}, (S,S)\rangle$



$\langle q^1, (x,\square), (x,\square), q^0_{xind}, (S,S)\rangle \quad \forall x \in \{0,\dots,9\}$



$\langle q_{xind}, (+,\square), (+,\square), q^0_{+ind}, (S,S)\rangle \quad \forall x \in \{0,\dots,9\}$



Dopo aver letto il +
Va a SINISTRA
Legge x=5 lo ricorda e
Continua (COME PRIMA)...

4 Alcune Definizioni

Che cos'è una parola?
Dato un alfabeto finito Σ, indichiamo l'insieme delle parole Σ^*, l'insieme delle sequenze costituite da un numero finito di elementi di Σ

$$\Sigma = <x_1 x_2 ... x_n> : n\mathbb{N}\forall 1 \leq i \leq n[x_i \in \Sigma]$$

Che cos'è uno Stato Globale?
Lo Stato Globale di una Macchina di Turing, in maniera informale, possiamo definirlo come la fotografia della macchina ad un certo istante. In modo formale, lo definiamo come la configurazione completa e istantanea della macchina, definita:

- dalla memoria attuale sul nastro, ovvero tutti i dati che sono stati scritti sul nastro finora, escludendo lo spazio vuoto (blank) all'esterno
- dalla posizione e contenuto della testina, ovvero la posizione dove sta puntando la testina sul nastro e qual è il simbolo che sta guardando in questo momento.
- dallo stato interno del controllo finito, ovvero lo stato in cui si trova la macchina in quell'istante.

Che cos'è una transizione?
Esiste una transizione da SG_1 a SG_2 se e solo se è possibile eseguire una delle istruzioni della macchina che ha come prerequisiti gli elementi di SG_1 e come risultato gli elementi di SG_2.

Che cos'è una computazione?
Una Computazione di una Macchina di Turing è l'intera sequenza di passi (detti Stati Globali) che la macchina compie, partendo dall'input iniziale fino a quando non raggiunge la fine (o non si ferma mai).

La Computazione è composta da varie fasi:

- *Inizio:* Lo stato Globale Iniziale è SG_0. Qui il nastro viene caricato con l'input x e la macchina si prepara all'azione. Quindi troviamo la testina posizionata sul primo simbolo dell'input.
- *Sviluppo:* Finché la macchina non ha finito, il calcolo procede in modo deterministico, ovvero ogni Stato Globale SG_i passa alla configurazione successiva SG_{i+1} eseguendo una quintupla.
- *Fine:* La Macchina può terminare in uno stato finale o non Terminare e si ha un ciclo.
    - Quando Termina in uno Stato Finale: Se la macchina raggiunge un passo k in cui il suo stato interno è uno stato finale, la com-

putazione si ferma immediatamente. La macchina non esegue più alcuna istruzione. Il risultato è quello che rimane scritto sul nastro.

* Quando non Termina e si avvia un ciclo infinito: Non tuitte le computazioni terminano, a volte la macchina può rimanere intrappolata in una sequenza infinita di stati (un ciclo) senza mai raggiungere uno stato finale.

5 Trasduttori e Riconoscitori

– **TRASDUTTORI**:

Una Macchina di Tipo Trasduttore, è una macchina che ha il compito di calcolare un valore. Solitamente si calcola il valore di una funzione. Questa macchina ha come output una sequenza di caratteri scritta sul nastro. Infatti lo stato finale non fornisce alcuna informazione sul suo esito. Quindi assumeremo che gli stati finali di una macchina di Tipo Trasduttore avranno un solo stato finale generico q_F.
Ogni Macchina di Tipo Trasduttore dispone di almeno 2 nastri:

* 1 per il nostro Input
* 1 per il nostro Output, dove verra scritto l'esito della computazione. Questo nastro è un nastro di sola scrittura, che si muove in una sola direzione (da sinistra a destra).

In realtà, per essere più precisi solitamente avremo una macchina a k nastri, dove 1 sarà dedicato all'output (come spiegato precedentemente), 1 sarà dedicato all'input e gli altri saranno quelli "di lavoro" che conterranno le operazioni che la macchina esegue e si annota.

– **RICONOSCITORI:**
Una Macchina di Tipo Riconoscitore ha il compito di decidere se l'input appartiene ad un insieme (es: problema visto precedentemente che costruiva una Macchina di Turing che decide se la stringa in input è palindroma). Questa macchina come output restituisce un valore booleano. Quindi possiamo affermare che lo stato finale fornisce informazioni sul suo esito. Infatti assumeremo che gli stati finali di una macchina di Tipo Riconoscitore avranno sempre un insieme di stati finali $\{q_A$ = Stato di Accettazione, q_R = Stato di Rigetto$\}$.

6 Modelli di Macchine di Turing e la loro Equivalenza

6.1 Macchine a Testine Indipendenti vs Macchine a Testine Solidali

Sia una Macchina di Turing T_k a k nastri con testine indipendenti, *esiste* una Macchina di Turing T' a $k+1$ nastri a testine solidali, *tale che*, *per ogni* input $x \in \Sigma^*$, l'output prodotto dalle due macchine sia identico.

Detto in altri termini:
$\forall T_k = \langle \Sigma, \Gamma, Q, q_0, F, \delta \rangle$ una MT a k nastri con testine indipendenti, *esiste* una macchina $T' = \langle \Sigma, \Gamma', Q', q_0', F', \delta' \rangle$ a $k+1$ nastri a testine solidali

$$: \forall x \in \Sigma^*, \quad out_{T_k}(x) = out_{T'}(x)$$

Costruiamo una macchina T_2 a testine indipendenti e una macchina T_{k+1} a testine solidali. Vediamo come si comportano



Figure 2

[Nella figura 2]: vediamo il caso di coincidenza dei movimenti – la MT a testine indipendenti T_2 esegue uno spostamento (D, D), simulabile direttamente da una MT a testine solidali T_3 con un unico movimento D.

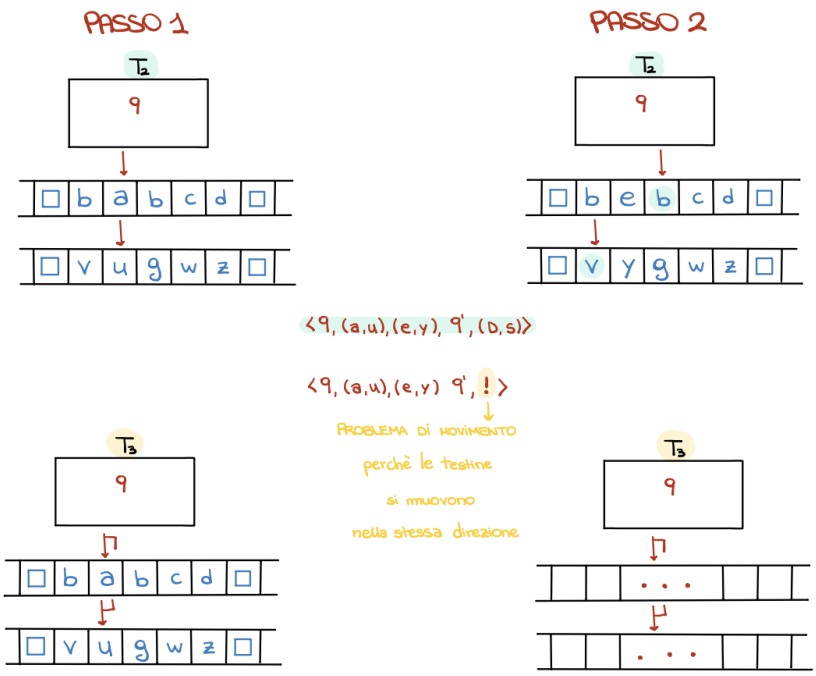


Figure 3

[Nella Figura 3]: Qui vediamo come la MT a testine indipendenti T_2 muove le testine dei due nastri con due diversi movimenti. La MT a testine solidali avrà un "problema di movimento".

Ora analizzeremo le computazione di due diverse Macchine di Turing, T_2 e T_3, operanti sul medesimo alfabeto Σ e partendo dalla medesima coppia di stringhe di input (x_1, x_2) dove $x_1 = babcdex_2 = vugw$. Analizzeremo come le due macchine gestiscono quello che prima abbiamo definito "problema di movimento".

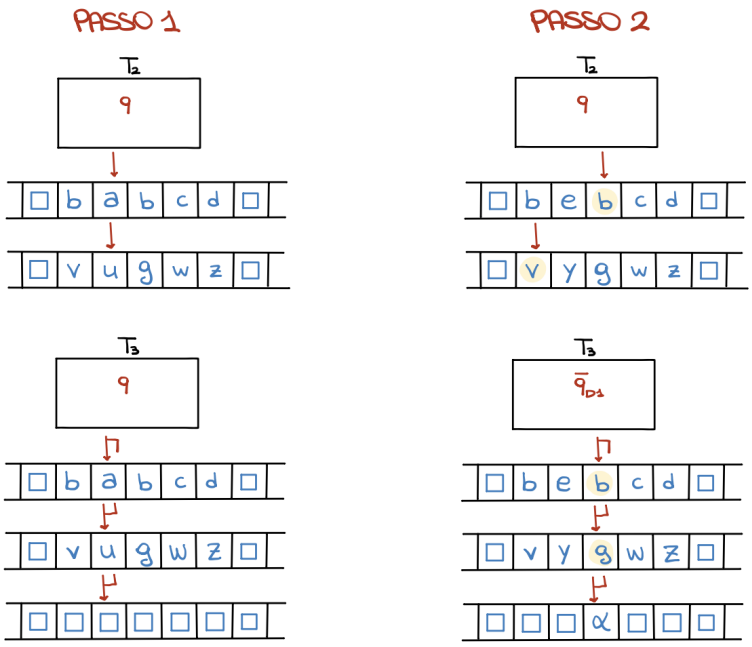


Figure 4

[Dalla figura 4]: Ci rendiamo conto del "mismatch". Possiamo risolverlo nel seguente modo:

Dal passo 3 e al passo 5 la macchina esegue una fase di riposizionamento, spostandosi verso l'estremità della parola mediante la seguente quintupla:

$$\langle \bar{q}_{D1}, (x, y), (x, y), \bar{q}_{D1}, D \rangle$$

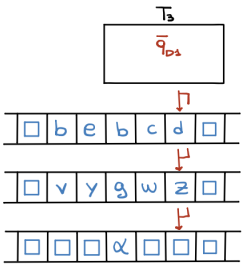


Figure 5:

$\langle \bar{q}_{D1}, (y, x), (\square, x), q_{D1}(y), S \rangle \forall x, y \in \Sigma$

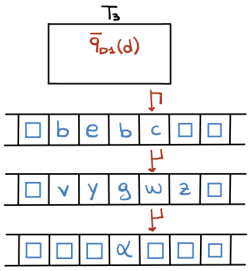


Figure 6:

$\langle q_{D1}(y), (z, x), (y, x), q_{D1}(z), S \rangle \forall x, y, z \in \Sigma$

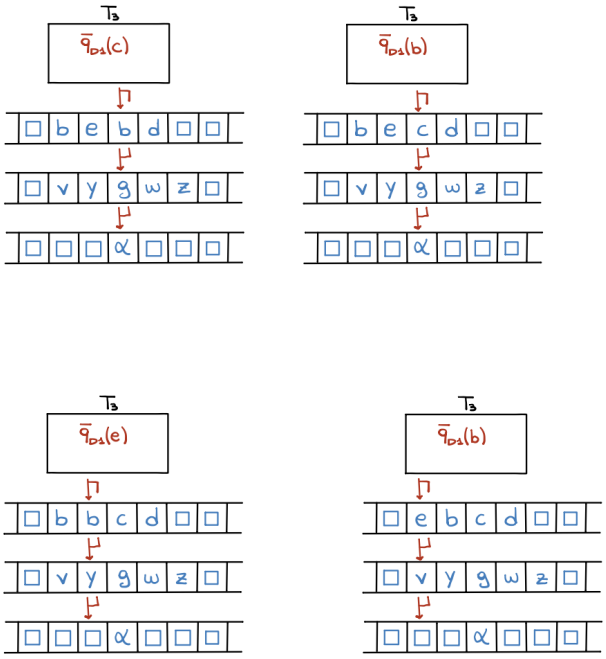


Figure 7: Per tutte queste vale la quintula descritta in figura 6

L'ultima macchina in figura 7 eseguirà la seguente quintupla

$$\langle q_{D1}(y), (z, x), (y, x), q_{D1}(z), S \rangle \forall x, y, z \in \Sigma$$

così facendo legge il blanck $\square$, torna a destra e inizia a spostare i caratteri verso Destra nel seguente modo:

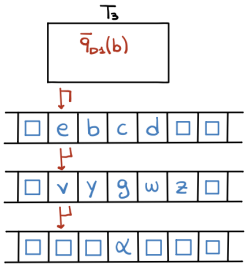


Figure 8

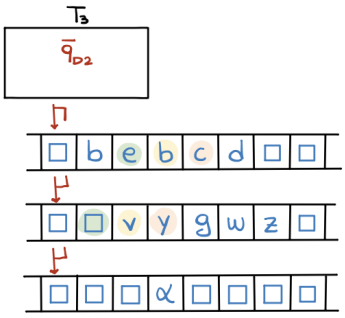


Figure 9:

$\langle \bar{q}_{D2}, (x, y), (x, \square), q_{D2}(y), D \rangle \forall x, y \in \Sigma$

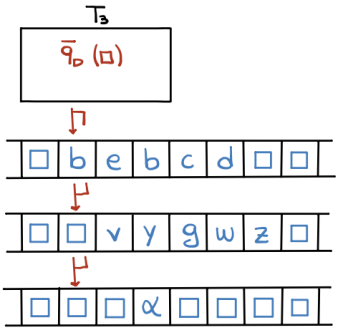


Figure 10:

$\langle q_{D2}(y), (x, y), (x, y), q_{D2}(z), D \rangle \forall x, y, z \in \Sigma$

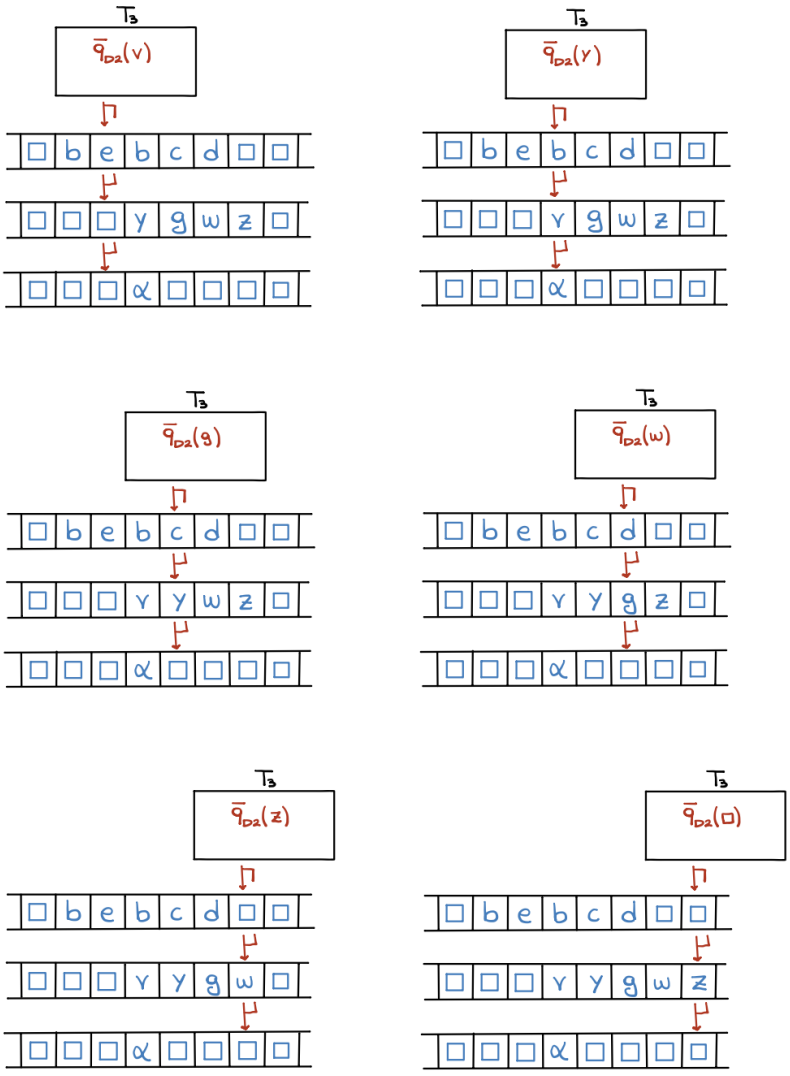


Figure 11: Per tutte queste vale la quintupla descritta in figura 10

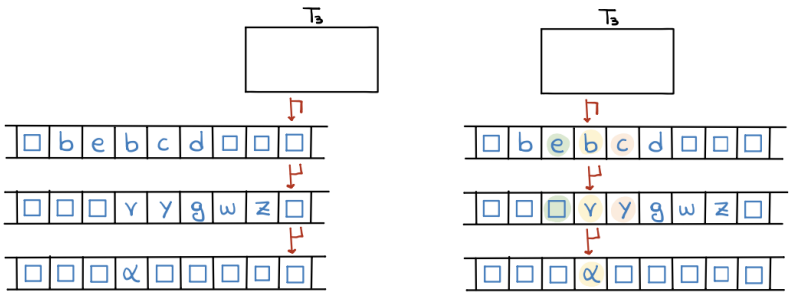


Figure 12

6.2 Macchina a k nastri vs Macchina ad 1 nastro

Sia una Macchina di Turing T_k a k nastri con testine solidali,
esiste una Macchina di Turing T' a 1 nastro, *tale che*,
per ogni input $x \in \Sigma^*$, l'output prodotto dalle due macchine sia identico.

Detto in altri termini:
$\forall T_k = \langle \Sigma, \Gamma, Q, q_0, F, \delta \rangle$ una MT a k nastri con testine solidali, *esiste* una macchina $T' = \langle \Sigma, \Gamma', Q', q_0', F', \delta' \rangle$ a 1 nastro

$$: \forall x \in \Sigma^*, \quad out_{T_k}(x) = out_{T'}(x)$$

Per simulare una macchina a k nastri T_k su un unico nastro T_1, andremo a costruire una macchina ad 1 nastro così composta: associamo a ogni cella un "indirizzo" i comune a tutti i nastri, formando così delle colonne di celle.
Se $(x_{i,1}, x_{i,2}, \ldots, x_{i,k})$ sono i caratteri scritti all'indirizzo i dei k nastri di T_k, questi vengono riportati sequenzialmente su T_1. In particolare, il simbolo $x_{i,j}$ del nastro j-esimo verrà scritto nella cella di T_1 in posizione $(i-1)k + j$.]
Ora, avendo la macchina Simuliamo il comportamento delle due macchine e vediamo come si comportano.

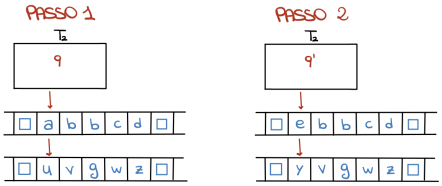


Figure 13: Mostra una MT a k nastri che esegue la seguente quintupla:

$$\langle q, (a,u), (e,y), q^{'}, F \rangle \forall x, y, z \in \Sigma$$

La macchina ad un unico nastro T_1, la cui computazione è equivalente a quella di T_k, viene configurata e opera secondo i seguenti criteri:

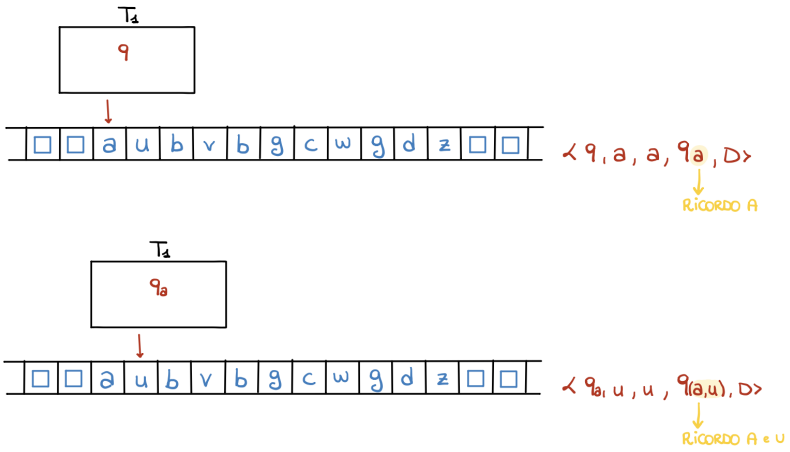


Figure 14: Mostra quando la MT ad 1 nastro si muove verso D per leggere e ricordare

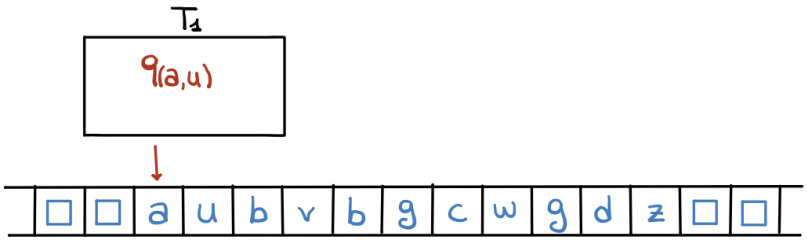


Figure 15: Successivamente la MT ad 1 nastro torna indietro al primo carattere, per iniziare a sovrascrivere

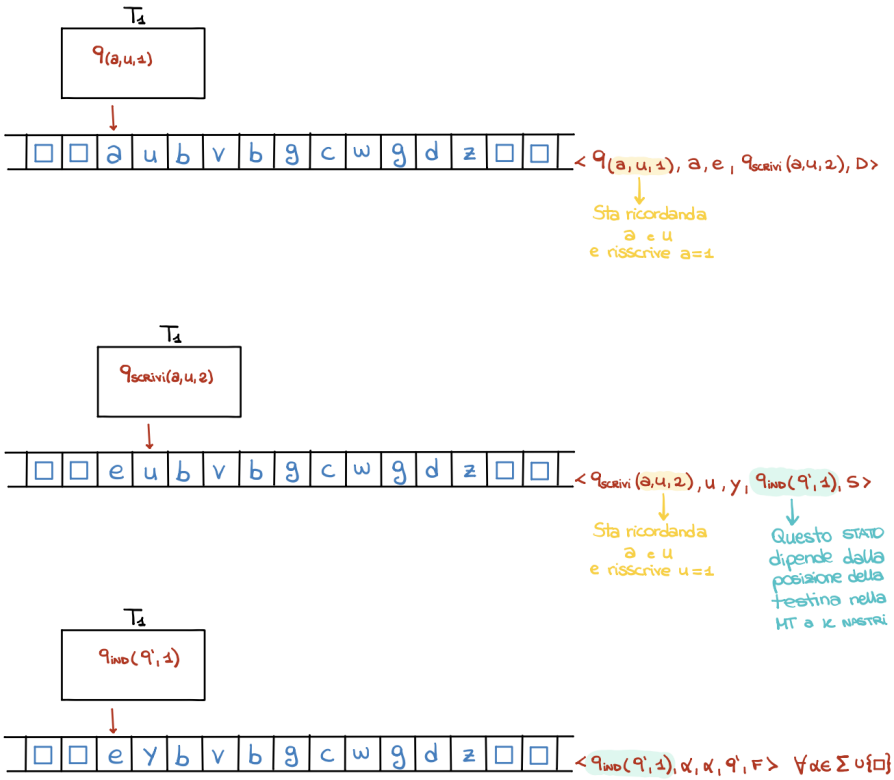


Figure 16: Mostra quando la MT ad 1 nastro si muove verso D per sovrascrivere e conclude posizionandosi in base al movimento della MT a k nastri

Dopo aver esaminato il caso in cui la macchina T_k resti ferma (F), analizziamo come T_1 simuli i movimenti a destra (D) e a sinistra (S)

- **CASO 1: MT a k nastri va verso Sinistra**

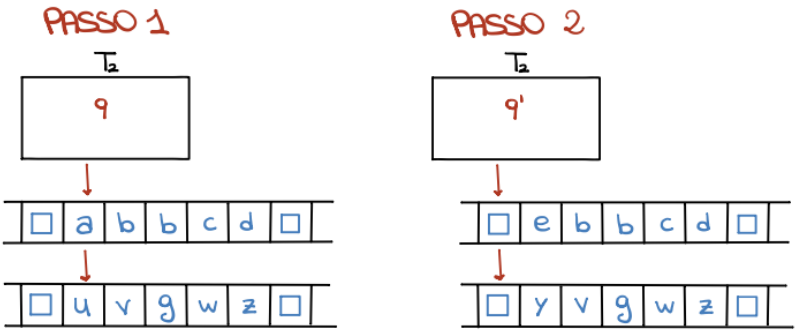


Figure 17: Mostra una MT a k nastri che esegue la seguente quintupla:

$$\langle q, (a, u), (e, y), q^{'}, S \rangle \forall x, y, z \in \Sigma$$

In questo caso, la computazione si svolgerà come visto prima nella figura 14, 15 e 16. Ma al penultimo passaggio entrerà nello stato $q_{IND}(q^{'}, 2)$ la quintupla finale sarà la seguente:

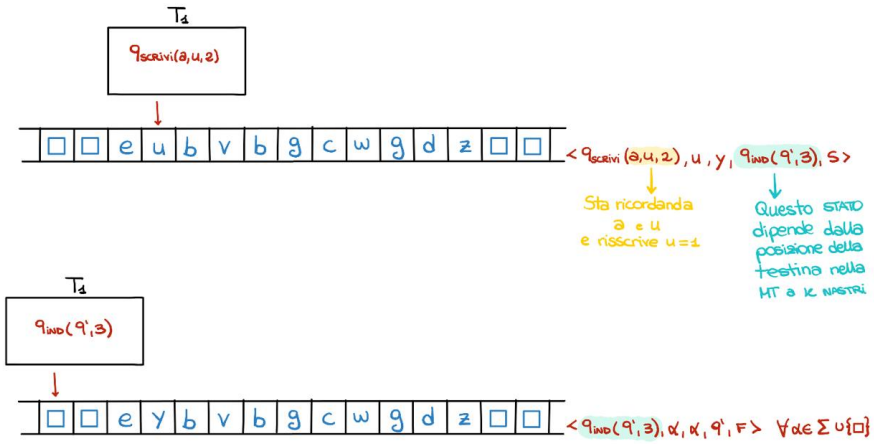


Figure 18

- **CASO 1: MT a k nastri va verso Destra**

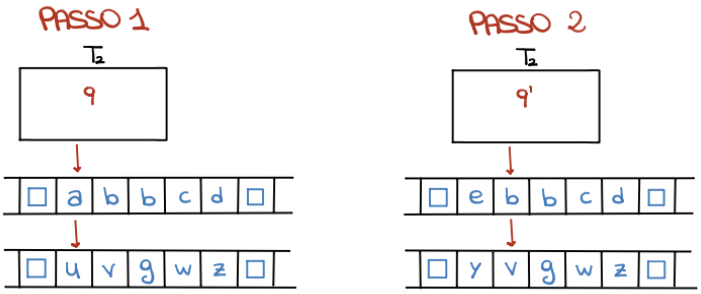


Figure 19: Mostra una MT a k nastri che esegue la seguente quintupla:

$$\langle q, (a, u), (e, y), q^{'}, D \rangle \forall x, y, z \in \Sigma$$

In questo caso, la computazione si svolgerà come visto prima nella figura 14, 15 e 16. Ma al penultimo passaggio entrerà nello stato $q(q^{'}, 1)$ la quintupla finale sarà la seguente:

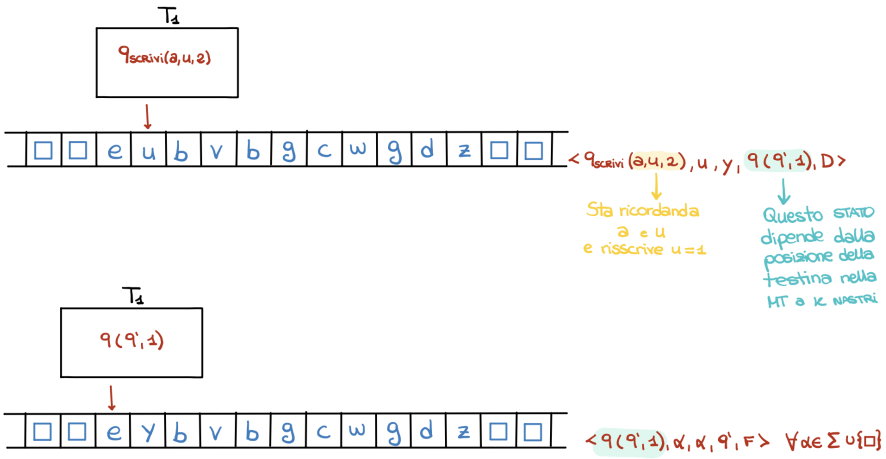


Figure 20

7 Struttura di P

Data un Macchina di Turing di tipo Riconoscitore ad un nastro. Definita su:

- un alfabeto Σ
- un insieme di stati Q
- e un iniseme di quintuple P che hanno lo scopo di definire le azioni che esegue la macchina.

Possiamo dire che una quintupla può essere vista come un'istruzione condizionale:

$$< q_1, s_1, s_2, q_2, mov >$$

Dove:

- if $\rightarrow q_1, s_1$
- then $\rightarrow q_2, s_2, mov$

Quindi P è una corrispondenza che associa elementi dell'insieme $Q \times \Sigma$ agli elementi dell'insieme $Q \times \Sigma \times \{dx, sx, mov\}$.
La corrispondenza P è totale poiché è possibile, avendo uno stato e un carattere, svolgere tutte le azioni possibili. Però non serve che sia totale poiché non andremo a scrivere tutte le quintuple perchè non tutte portano ad un risultato corretto.

8 Macchine di Turing Deterministiche e Non Deterministiche

- Una **Macchina Deterministica** è una macchina di Turing in cui ogni situazione della macchina ha un'unica istruzione possibile. Ovvero, per ogni coppia (stato attuale, simbolo letto) esiste al massimo una quintupla che indica cosa fare. Quindi, quando la macchina legge un simbolo in un certo stato, non ha scelta: sa esattamente quale simbolo scrivere, in quale stato andare e in che direzione muovere la testina. Il comportamento della macchina è quindi completamente determinato dal suo insieme di istruzioni non può "scegliere" tra più possibilità.
- Una **Macchina Non Deterministica** è una macchina è una macchina di Turing in cui possono esistere più istruzioni valide per la stessa coppia (stato, simbolo). Cioè la macchina, quando si trova in una certa configurazione, può avere più possibili mosse. Ogni mossa possibile dà origine a una nuova ramificazione di computazione. La computazione può quindi essere vista come un albero, dove ogni nodo rappresenta uno stato globale e ogni figlio una scelta possibile. Cosa vuol dire "non deterministica"? Significa che la macchina può "scegliere" fra più strade. Non sappiamo quale prenderà, quindi consideriamo tutte le possibilità contemporaneamente.

$$o_{NT(x)} = \begin{cases} \text{q}_A \text{ se almeno una delle computazioni deterministiche di NT(x) termina in stato di acc.} \\ \text{q}_R \text{ se tutte le computazioni deterministiche terminano in uno stato di rifiuto} \\ \text{non definito altrimenti} \end{cases}$$

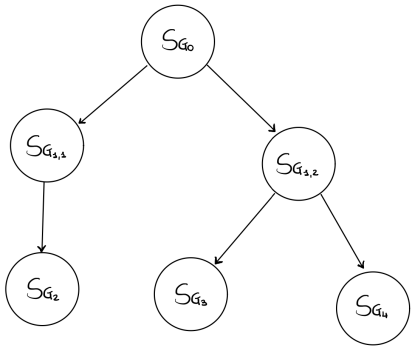


Figure 21: Albero che rappresenta una computazione Non Deterministica

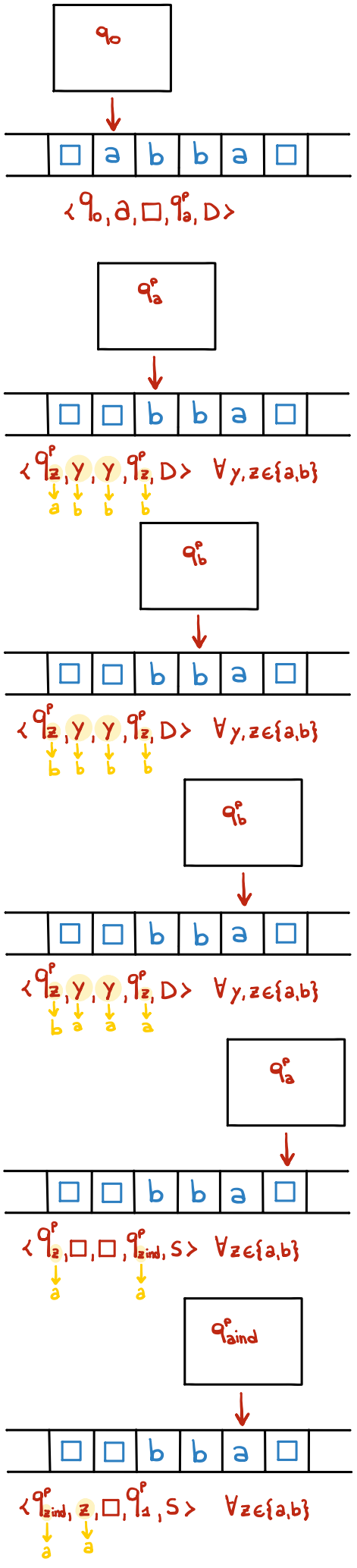
9 Esercitazione:

Esercizio Palindroma e Pari:
Voglio definire se una parola è palindroma (ovvero se rimane invariata leggendola da entrambi i versi) ed è Pari.

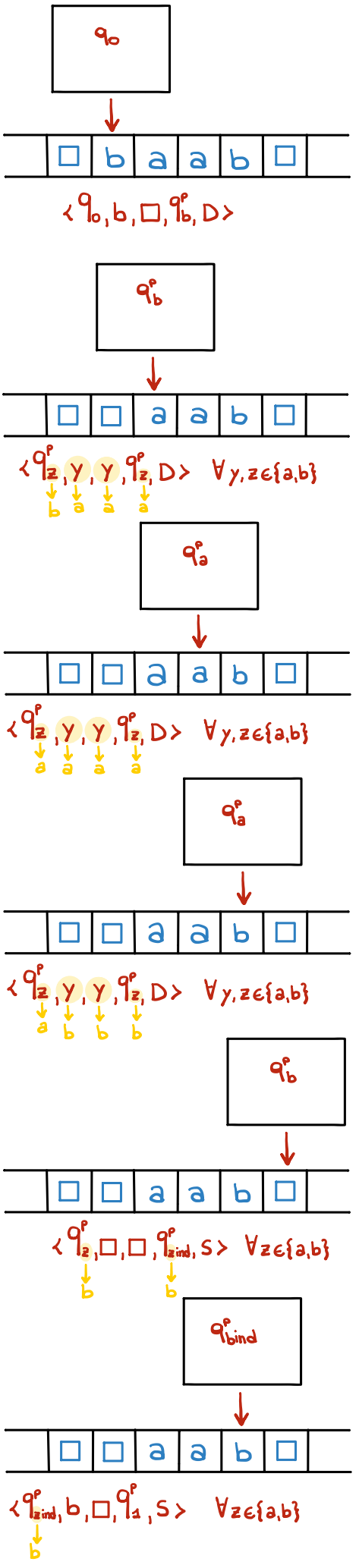
Definiamo la Macchina di Turing:

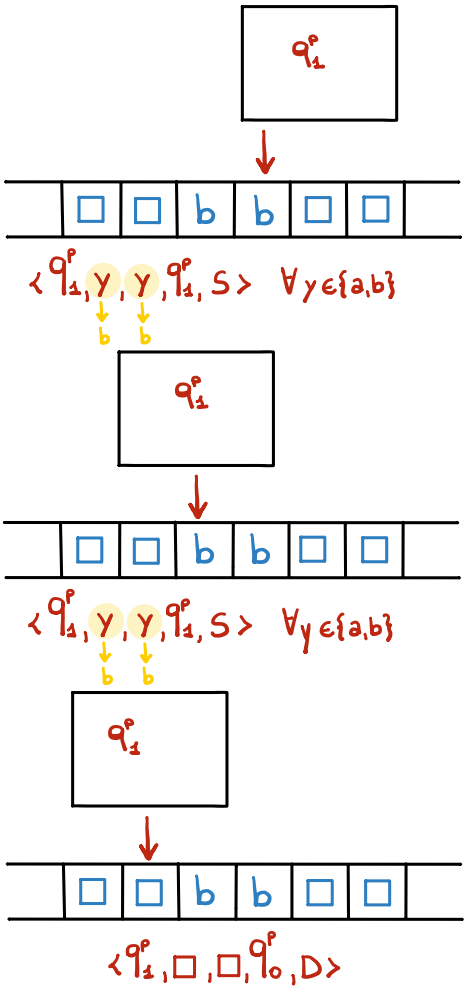
- L'alfabeto: $\Sigma = \{a, b\}$
- Lo stato Iniziale: q_0
- L'insieme degli Stati: $\{q_0, q_z^p, q_{zIND}^p, q_1^P, q_0^P, q_{PPAL}^P\}$
- L'insieme degli stati Finali: $\{q_{PPAL}^P\}$
- Le Quintuple: *Illustrate nelle pagine seguenti nell'esempio riportato*

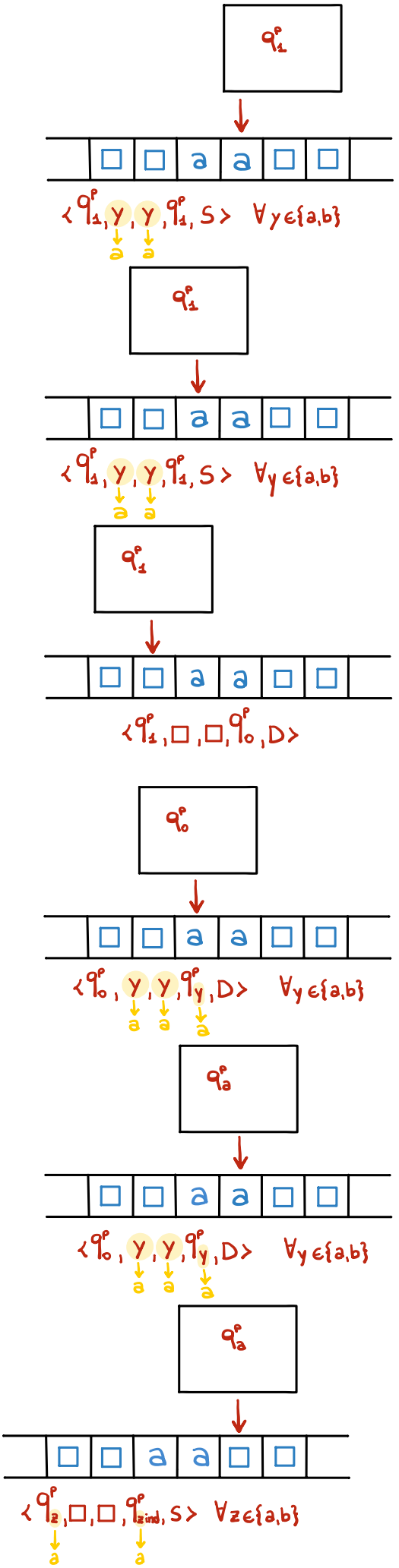
PAROLA_1: abba

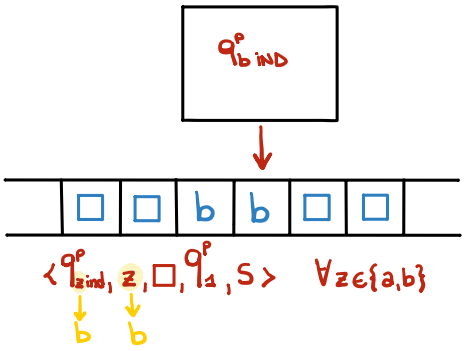


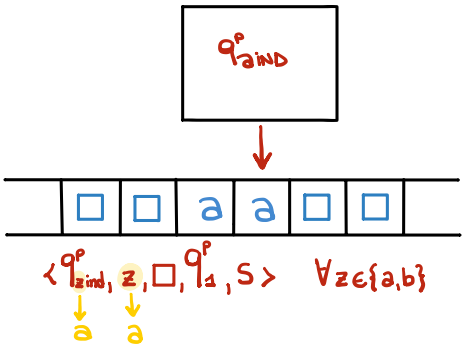
PAROLA_2: baab

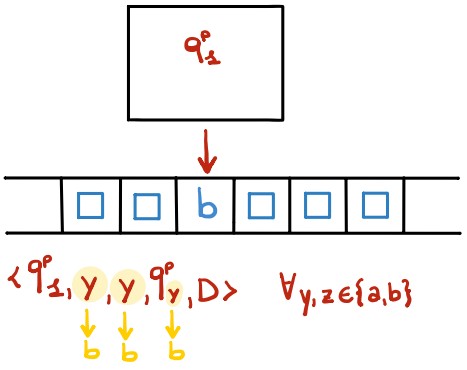


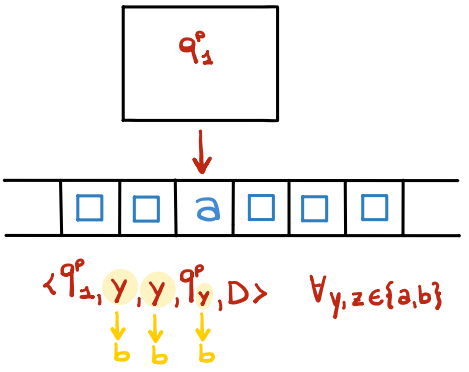

q_1^P
□ □ b b □ □
$\langle q_1^P, y, y, q_1^P, S\rangle \quad \forall y \in \{a,b\}$
b b
q_1^P
□ □ b b □ □
$\langle q_1^P, y, y, q_1^P, S\rangle \quad \forall y \in \{a,b\}$
b b
q_1^P
□ □ b b □ □
$\langle q_1^P, \square, \square, q_0^P, D\rangle$
q_0^P
□ □ b b □ □
$\langle q_0^P, y, y, q_y^P, D\rangle \quad \forall y \in \{a,b\}$
b b b
q_b^P
□ □ b b □ □
$\langle q_z^P, y, y, q_y^P, D\rangle \quad \forall y,z \in \{a,b\}$
b b b b
q_b^P
□ □ b b □ □
$\langle q_z^P, \square, \square, q_{z\text{ind}}^P, S\rangle \quad \forall z \in \{a,b\}$
b b

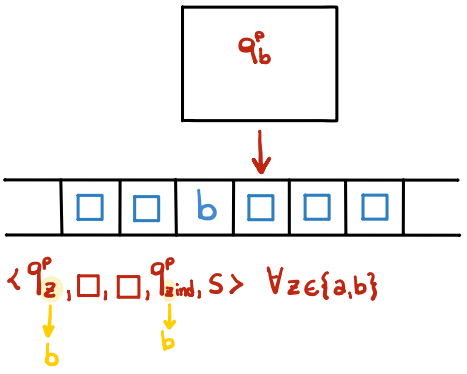

q_1^P
□ □ a a □ □
$\langle q_1^P, y, y, q_1^P, S\rangle \quad \forall y \in \{a,b\}$
a a
q_1^P
□ □ a a □ □
$\langle q_1^P, y, y, q_1^P, S\rangle \quad \forall y \in \{a,b\}$
a a
q_1^P
□ □ a a □ □
$\langle q_1^P, \square, \square, q_0^P, D\rangle$
q_0^P
□ □ a a □ □
$\langle q_0^P, y, y, q_y^P, D\rangle \quad \forall y \in \{a,b\}$
a a a
q_a^P
□ □ a a □ □
$\langle q_0^P, y, y, q_y^P, D\rangle \quad \forall y \in \{a,b\}$
a a a
q_a^P
□ □ a a □ □
$\langle q_z^P, \square, \square, q_{z\text{ind}}^P, S\rangle \quad \forall z \in \{a,b\}$
a a

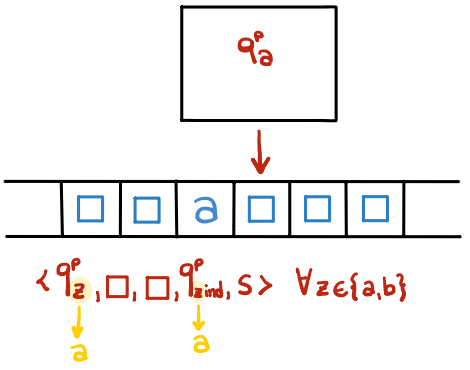

$q^p_{b\,IND}$
□ □ b b □ □
$\langle q^p_{z\,ind}, z, \square, q^p_1, S\rangle \quad \forall z \in \{a,b\}$
b b

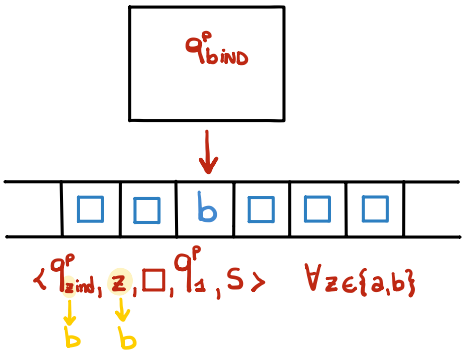

$q^p_{a\,IND}$
□ □ a a □ □
$\langle q^p_{z\,ind}, z, \square, q^p_1, S\rangle \quad \forall z \in \{a,b\}$
a a

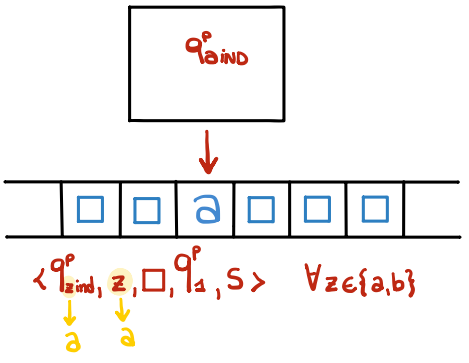

q^p_1
□ □ b □ □ □
$\langle q^p_1, y, y, q^p_y, D\rangle \quad \forall y,z \in \{a,b\}$
b b b

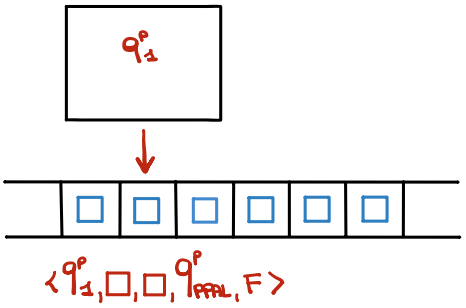

q^p_1
□ □ a □ □ □
$\langle q^p_1, y, y, q^p_y, D\rangle \quad \forall y,z \in \{a,b\}$
b b b

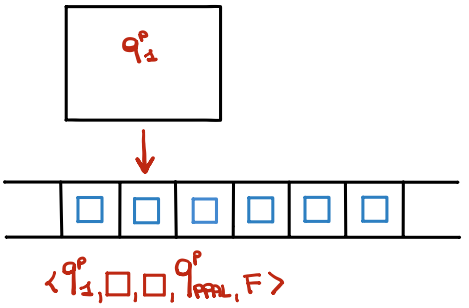

q^p_b
□ □ b □ □ □
$\langle q^p_2, \square, \square, q^p_{z\,ind}, S\rangle \quad \forall z \in \{a,b\}$
b b


q^p_a
□ □ a □ □ □
$\langle q^p_2, \square, \square, q^p_{z\,ind}, S\rangle \quad \forall z \in \{a,b\}$
a a


$q^p_{b\,IND}$
□ □ b □ □ □
$\langle q^p_{z\,ind}, z, \square, q^p_1, S\rangle \quad \forall z \in \{a,b\}$
b b


$q^p_{a\,IND}$
□ □ a □ □ □
$\langle q^p_{z\,ind}, z, \square, q^p_1, S\rangle \quad \forall z \in \{a,b\}$
a a


q^p_1
□ □ □ □ □ □
$\langle q^p_1, \square, \square, q^p_{FFAL}, F\rangle$

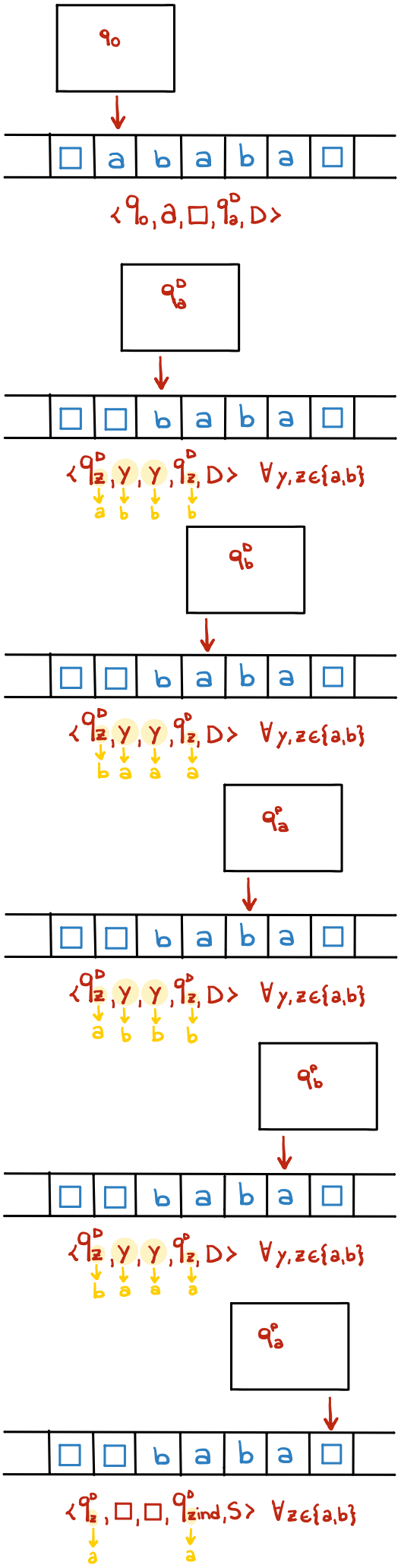

q^p_1
□ □ □ □ □ □
$\langle q^p_1, \square, \square, q^p_{FFAL}, F\rangle$

Esercizio Palindroma e Dispari:
Voglio definire se una parola è palindroma (ovvero se rimane invariata leggendola da entrambi i versi) ed è Dispari.

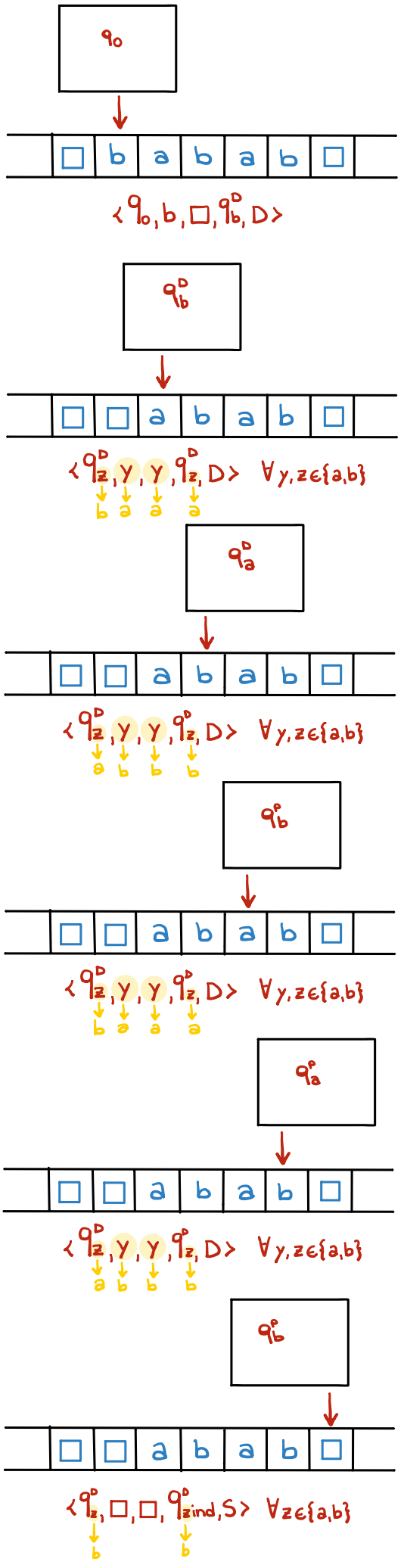
Definiamo la Macchina di Turing:

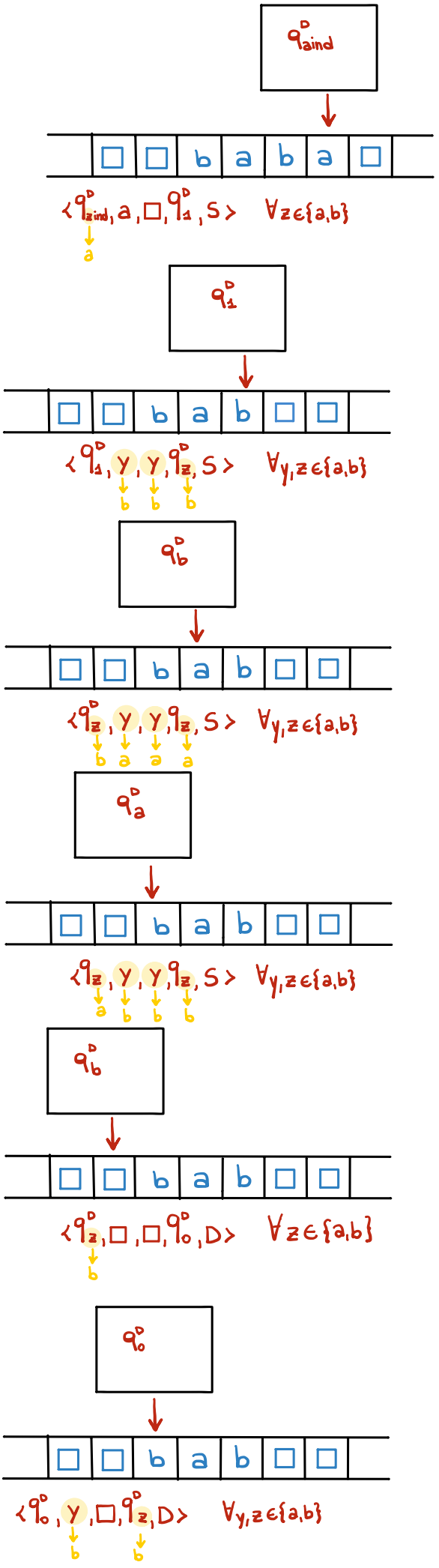
- L'alfabeto: $\Sigma = \{a, b\}$
- Lo stato Iniziale: q_0
- L'insieme degli Stati: $\{q_0, q_z^D, q_{zIND}^D, q_1^D, q_0^D, q_{DPAL}^D\}$
- L'insieme degli stati Finali: $\{q_{DPAL}^D\}$
- Le Quintuple: *Illustrate nelle pagine seguenti nell'esempio riportato*

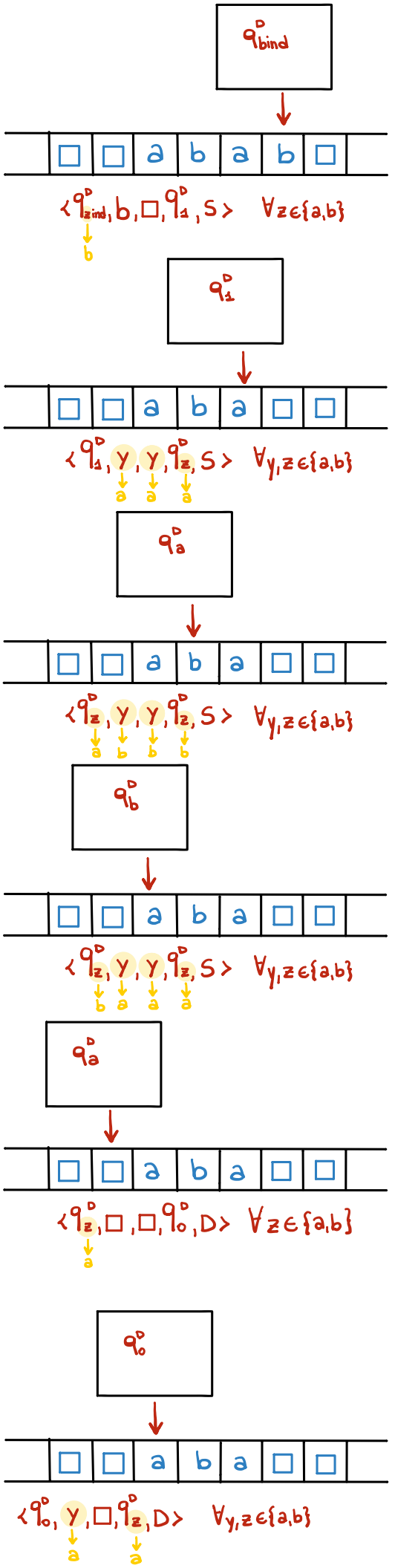
PAROLA_1: ababa

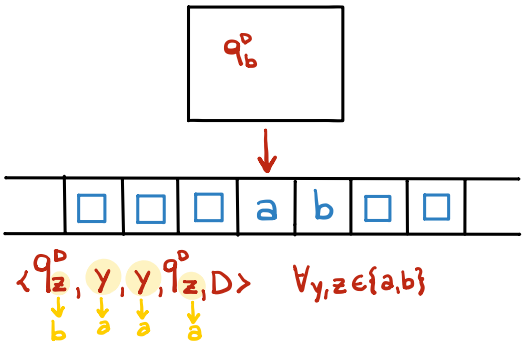


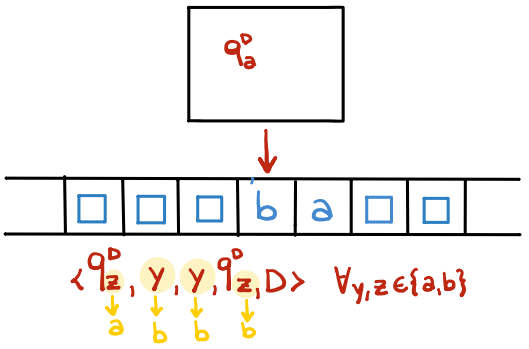
PAROLA_2: babab

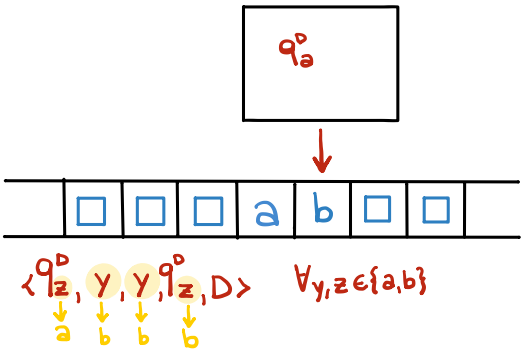


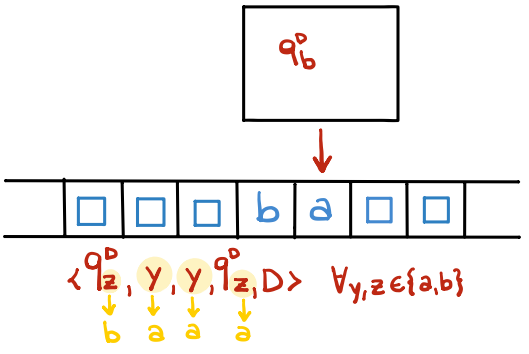

q_{aind}
□ □ b a b a □
$\langle q_{aind}, a, \square, q_1, S \rangle \quad \forall z \in \{a,b\}$
q_1
□ □ b a b □ □
$\langle q_1, y, y, q_z, S \rangle \quad \forall y,z \in \{a,b\}$
q_b
□ □ b a b □ □
$\langle q_z, y, y, q_z, S \rangle \quad \forall y,z \in \{a,b\}$
q_a
□ □ b a b □ □
$\langle q_z, y, y, q_z, S \rangle \quad \forall y,z \in \{a,b\}$
q_b
□ □ b a b □ □
$\langle q_z, \square, \square, q_o, D \rangle \quad \forall z \in \{a,b\}$
q_o
□ □ b a b □ □
$\langle q_o, y, \square, q_z, D \rangle \quad \forall y,z \in \{a,b\}$

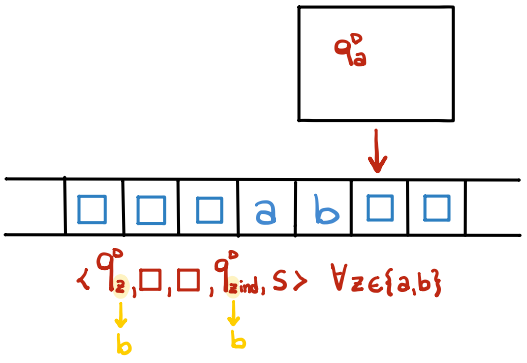

q_{bind}
□ □ a b a b □
$\langle q_{zind}, b, \square, q_1, S \rangle \quad \forall z \in \{a,b\}$
q_1
□ □ a b a □ □
$\langle q_1, y, y, q_z, S \rangle \quad \forall y,z \in \{a,b\}$
q_a
□ □ a b a □ □
$\langle q_z, y, y, q_z, S \rangle \quad \forall y,z \in \{a,b\}$
q_b
□ □ a b a □ □
$\langle q_z, y, y, q_z, S \rangle \quad \forall y,z \in \{a,b\}$
q_a
□ □ a b a □ □
$\langle q_z, \square, \square, q_o, D \rangle \quad \forall z \in \{a,b\}$
q_o
□ □ a b a □ □
$\langle q_o, y, \square, q_z, D \rangle \quad \forall y,z \in \{a,b\}$

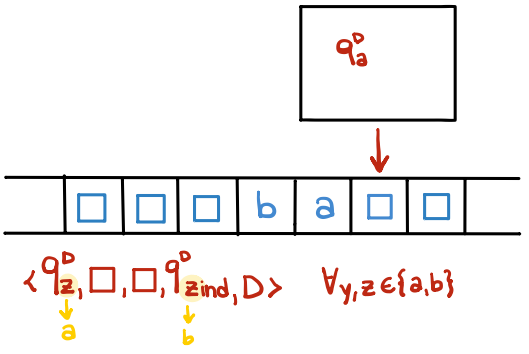

q^D_b
□ □ □ a b □ □
$\langle q^D_z, y, y, q^D_z, D\rangle$ $\forall y,z\in\{a,b\}$
b a a a

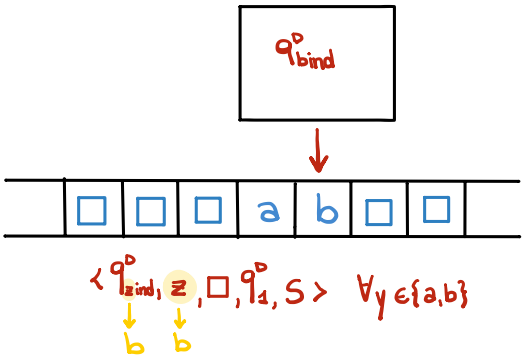

q^D_a
□ □ □ b a □ □
$\langle q^D_z, y, y, q^D_z, D\rangle$ $\forall y,z\in\{a,b\}$
a b b b

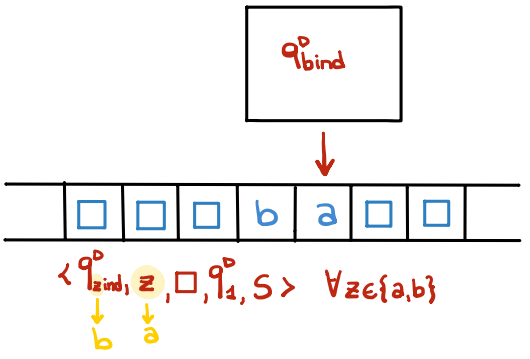

q^D_a
□ □ □ a b □ □
$\langle q^D_z, y, y, q^D_z, D\rangle$ $\forall y,z\in\{a,b\}$
a b b b

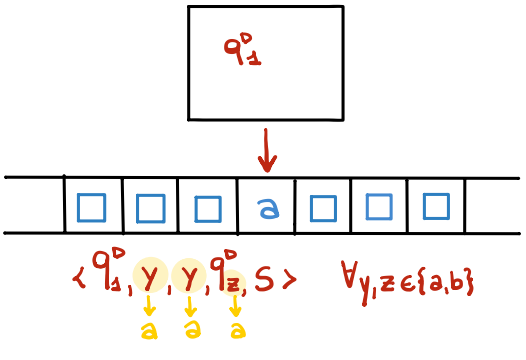

q^D_b
□ □ □ b a □ □
$\langle q^D_z, y, y, q^D_z, D\rangle$ $\forall y,z\in\{a,b\}$
b a a a

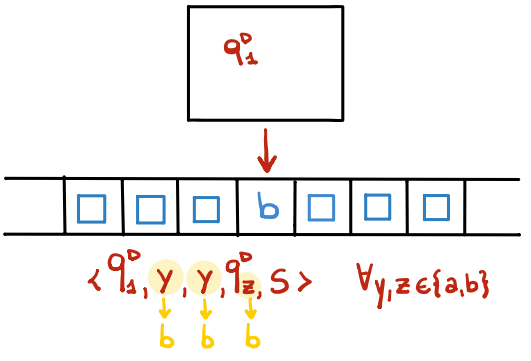

q^D_a
□ □ □ a b □ □
$\langle q^D_z, \square, \square, q^D_{z\text{ind}}, S\rangle$ $\forall z\in\{a,b\}$
b b

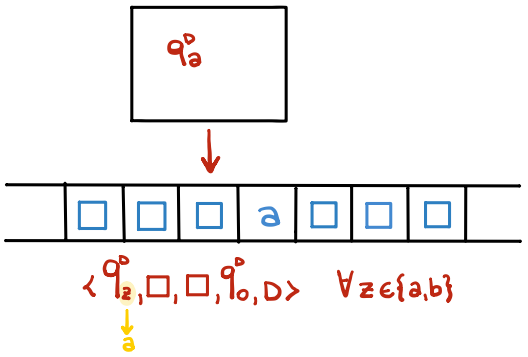

q^D_a
□ □ □ b a □ □
$\langle q^D_z, \square, \square, q^D_{z\text{ind}}, D\rangle$ $\forall y,z\in\{a,b\}$
a b

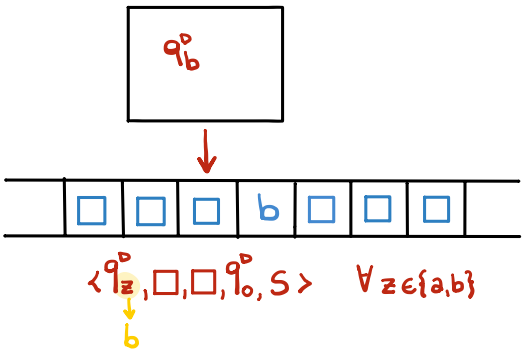

q^D_{bind}
□ □ □ a b □ □
$\langle q^D_{z\text{ind}}, z, \square, q^D_1, S\rangle$ $\forall y\in\{a,b\}$
b b

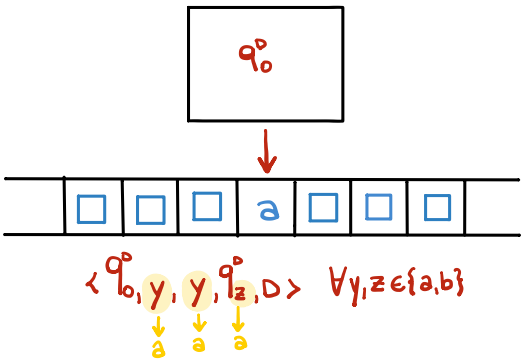

q^D_{bind}
□ □ □ b a □ □
$\langle q^D_{z\text{ind}}, z, \square, q^D_1, S\rangle$ $\forall z\in\{a,b\}$
b a

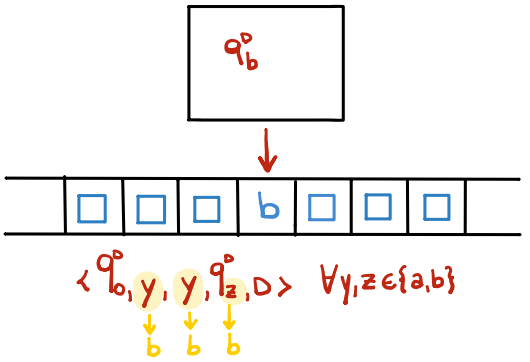

q^D_1
□ □ □ a □ □ □
$\langle q^D_1, y, y, q^D_z, S\rangle$ $\forall y,z\in\{a,b\}$
a a a

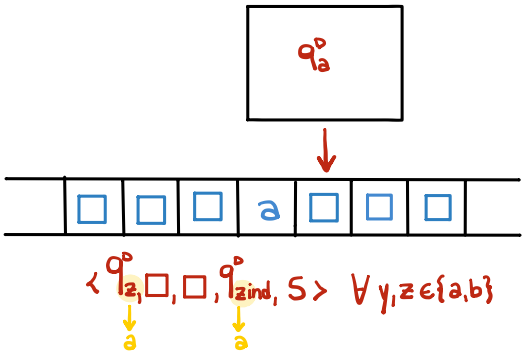

q^D_1
□ □ □ b □ □ □
$\langle q^D_1, y, y, q^D_z, S\rangle$ $\forall y,z\in\{a,b\}$
b b b

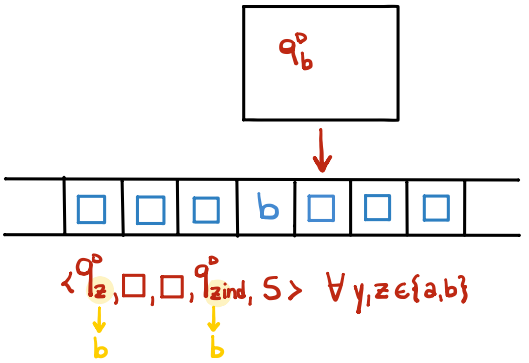

q^0_a
a
$\langle q^0_z, \square, \square, q^0_o, D \rangle \quad \forall z \in \{a,b\}$
a

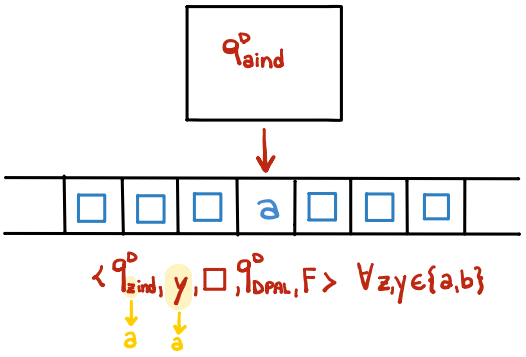

q^0_b
b
$\langle q^0_z, \square, \square, q^0_o, S \rangle \quad \forall z \in \{a,b\}$
b

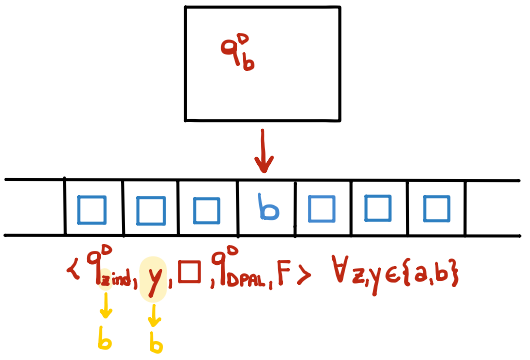

q^0_o
a
$\langle q^0_o, y, y, q^0_z, D \rangle \quad \forall y,z \in \{a,b\}$
a a a


q^0_b
b
$\langle q^0_o, y, y, q^0_z, D \rangle \quad \forall y,z \in \{a,b\}$
b b b


q^0_a
a
$\langle q^0_z, \square, \square, q^0_{zind}, S \rangle \quad \forall y,z \in \{a,b\}$
a a


q^0_b
b
$\langle q^0_z, \square, \square, q^0_{zind}, S \rangle \quad \forall y,z \in \{a,b\}$
b b


q^0_{aind}
a
$\langle q^0_{zind}, y, \square, q^0_{DPAL}, F \rangle \quad \forall z,y \in \{a,b\}$
a a


q^0_b
b
$\langle q^0_{zind}, y, \square, q^0_{DPAL}, F \rangle \quad \forall z,y \in \{a,b\}$
b b

Esercizio Palindroma:
Voglio definire se una parola è palindroma (ovvero se rimane invariata leggendola da entrambi i versi).
Svolgiamo l'esercizio a Scatola Chiusa.

Definiamo la Macchina di Turing: T_{PAL} : $inputx \in \Sigma = \{a, b\}^*$ sul nastro N1

FASE 1) Copia il contenuto di N1 su N2 e N3

FASE 2) Simula T_{PPAL} su N2:
se $T_{PPAL}(x)$ accetta allora termina in q_A,
altrimenti se $T_{PPAL}(x)$ rigetta allora esegui FASE 3

FASE 3) Simula T_{DPAL} su N3
se $T_{DPAL}(x)$ accetta allora termina in q_A,
altrimenti termina in q_R

10 Riduciamo la cardinalità di Σ

Qualsiasi Macchina di Turing (MT) che opera su un alfabeto qualsiasi Σ (con almeno due simboli), può essere simulata da una Macchina di Turing equivalente T_{01} che usa solo l'alfabeto binario $\Sigma = \{0, 1\}$.
Sia T una Macchina di Turing definita su un alfabeto Σ, dove $|\Sigma| \geq 2$.
Sia k il numero minimo di bit necessario per rappresentare ogni simbolo di Σ. Questo valore è dato da $k = \lceil log_2 n \rceil$.
L'alfabeto Σ è ordinato come $\Sigma = \{\sigma_0, \sigma_1, , \ldots, \sigma_{\Sigma 1\}}$.
Ogni simbolo $\sigma \in \Sigma$ viene codificato con una sequenza binaria di lunghezza k:

$$b(\sigma) = (b_1(\sigma), b_2(\sigma), \ldots, b_k(\sigma))$$

Esempio: Se $\Sigma = \{a, b, c\}$, allora $|\Sigma| = 3$. Si ha $k = \lceil \log_2 3 \rceil = 2$.

- a potrebbe essere codificato con 00.
- b potrebbe essere codificato con 01.
- c potrebbe essere codificato con 10.

La Macchina di Turing equivalente T_{01} usa l'alfabeto $\Sigma_{01} = \{0, 1\}$.
Quando la Macchina T (quella originale) legge o scrive un simbolo σ sul suo nastro, la macchina T_{01} deve leggere o scrivere la corrispondente sequenza di k bit (cioè $b(\sigma)$) sul suo nastro.
La simulazione è realizzata costruendo un insieme di stati Q_{01} e un insieme di quíntuple (transizioni) P_{01} per T_{01} che emulano il comportamento di T.

11 Linguaggi e Macchine di Turing

Considerando una Macchina di Turing di tipo riconoscitore e sia dato un input $x \in \Sigma^*$. Questa macchina verifica se x soddisfa una certa proprietà, o predicato. Indicheremo questo con $\pi(.)$. Se indichiamo con

$$o_T(x) = q_A \Leftrightarrow \pi(x),$$

dove:

- q_A è lo stato di accettazione
- $\pi(x)$ è il predicato che, quando vero, garantisce l'accettazione.

O equivalentemente possiamo scrivere

$$\{x \in \Sigma^* : o_T(x) = q_A\} = \{x \in \Sigma^* : \pi(x)\}$$

- **ALCUNE DEFINIZIONI:**

Un **linguaggio L** è un sottoinsieme di $\Sigma^* : L \subseteq \Sigma^*$

Esempio: Sia l'alfabeto $\Sigma = \{0, 1\}$. Σ^* è l'insieme di tutte le stringhe binarie (es. $0, 1, 00, 01, 10, 11, \ldots$).
Definisci L un linguaggio formato da tutte le stringhe binarie con un numero pari di '1'.

Le parole saranno $\rightarrow 0, 001100, 101, ecc..$

Un **Linguaggio Complemento $\mathbf{L^c}$** di un linguaggio $L \subseteq \Sigma^*$ è l'insieme delle parole non contenute in $L : L^c = \Sigma^* - L$

Esempio: Sia $\Sigma = \{0, 1\}$ e $L =$ stringhe con un numero pari di '1', il suo complemento L^c è:

$$L^c = \{x \in \Sigma^* : x \text{ ha un numero dispari di '1'}\}$$

Le parole saranno $\rightarrow 1, 010, 11100, 0001000, ecc..$

N.B: Dato un qualsiasi $L \subseteq \Sigma^*$ non è banale progettare una Macchina di Turing che accetti tutte le parole in L.

Un **linguaggio $\mathbf{L \subseteq \Sigma^*}$** è **accettabile** se esiste una Macchina di Turing T tale che:

- Se $x \in L$, allora $T(x)$ termina in uno stato di accettazione (q_A).
- Se $x \notin L$, allora $T(x)$ termina in uno stato di rifiuto (q_R) **oppure** non termina mai (loop infinito).

In altre parole: $x \in L \iff T(x) = q_A$. Se $x \notin L$, l'esito è q_R oppure indefinito.

Un **linguaggio $\mathbf{L \subseteq \Sigma^*}$** è **decidibile** se esiste una Macchina di Turing T che *termina per ogni input* $x \in \Sigma^*$ (definita come "decisore") e tale che:

- $x \in L \implies T(x) = q_A$
- $x \notin L$ o $x \in L^c \implies T(x) = q_R$

Esempio:

Consideriamo il linguaggio $L_{bin_palindroma}$:

$$L_{bin_palindroma} = \{x \in \Sigma^* \mid x \text{ è una stringa binaria palindroma}\}$$

dove $\Sigma = \{0, 1\}$

Esempi: $0, 1, 010, 11, 0110, 10001, \ldots$

Macchina di Turing T per $L_{bin_palindroma}$

La Macchina di Turing T opera come segue:

1. T confronta il primo simbolo letto sulla stringa d'ingresso con l'ultimo simbolo letto non ancora marcato.
2. Se i due simboli sono uguali, T li cancella o li marca (es. sostituendoli con un simbolo speciale, come $\square$) e ripete il processo sulla parte rimanente della stringa.
3. Se la stringa che rimane sul nastro è vuota (ϵ) o contiene un solo simbolo, la Macchina T **accetta** lo stato q_A.
4. Se, in qualsiasi momento, i simboli confrontati sono diversi, la Macchina T **rifiuta** lo stato q_R.

Poiché per ogni input $x \in \Sigma^*$ la Macchina di Turing T termina sempre, raggiungendo uno stato di accettazione (q_A) se $x \in L_{bin_palindroma}$ o uno stato di rifiuto (q_R) se $x \notin L_{bin_palindroma}$, si conclude che:

Il linguaggio $L_{bin_palindroma}$ è <u>Decidibile</u>.

Teorema: Un linguaggio $L \subseteq \Sigma^*$ è decidibile se e soltanto se L e L^c sono accettabili.

DIM 1:
$\implies$) **Se L è Decidibile, allora L e L^c sono Accettabili**
Se L è decidibile, allora esiste una macchina di Turing che per ogni $x \in \Sigma^*$, con input x:

- termina nello stato q_A se $x \in L$
- termina nello stato q_R se $x \in L^c$.

Dobbiamo trovare una macchina T' che accetti L^c.
Deriviamo da T una macchina T'.
Gli stati di T' sono uguali a quelli di T, aggiungiamo come stati finali q'_A, q'_R.
Quindi, aggiungeremo 2 nuove quintuple:

- $\langle q_A, x, x, q'_R, \text{stop}\rangle$: Se T termina in q_A (cioè, $x \in L$), T' esegue una transizione finale che la porta in un nuovo stato di **rigetto** (q'_R).
- $\langle q_R, x, x, q'_A, \text{stop}\rangle$: Se T termina in q_R (cioè, $x \in L^c$), T' esegue una transizione finale che la porta in un nuovo stato di **accettazione** (q'_A).

Concludiamo affermando che, T' accetta x se e soltanto se T rigetta x.
Poiché T rigetta x se e solo se $x \in L^c$, la macchina T' accetta il linguaggio L^c.
Se L è decidibile, abbiamo trovato una macchina (T) che accetta L e una macchina (T') che accetta L^c.
Dunque, sia L che L^c sono accettabili.

Caratteristica	**Macchina T (Decide L)**	**Macchina T' (Accetta L^c)**
Scopo	Decidere L. Termina sempre in q_A o q_R.	Accettare L^c. Può non terminare se non fosse decidibile, ma T' eredita la terminazione di T.
Insieme degli Stati	Stati originali Stati(T)	Stati(T) $\cup \{q'_A, q'_R\}$
Stato di Accettazione Finale	q_A	q'_A
Stato di Rigetto Finale	q_R	q'_R
Regole di Transizione	Quintuple originali di T: Δ_T	Δ_T + **le due nuove regole:**
Nuove Regole Aggiunte	Nessuna	$\langle q_A, u, u, q'_R, \text{ferma}\rangle$ $\langle q_R, u, u, q'_A, \text{ferma}\rangle$ (Per ogni simbolo $u \in \Sigma \cup \{\text{Blank}\}$)
Comportamento se $x \in L$	$T(x)$ termina in q_A (Accetta)	$T'(x)$ termina in q'_R (Rigetta)
Comportamento se $x \in L^c$	$T(x)$ termina in q_R (Rigetta)	$T'(x)$ termina in q'_A (Accetta)

DIM 2:
$\Longleftarrow$) Se L e L^c sono accettabili allora L è decidibile

Se L è accettabile, allora esiste una macchina di Turing che per ogni $x \in \Sigma^*$ la computazione finale della macchina T sull'input x termina in q_A se $x \in L$.
In altre parole: $\exists T : \forall x \in \Sigma^*[o_T(x) = q_A \Leftrightarrow x \in L]$.

Se L^c è accettabile, allora esiste una macchina di Turing che per ogni $x \in \Sigma^*$ la computazione finale della macchina T sull'input x termina in q_A se $x \in L$. In altre parole: $\exists T^c : \forall x \in \Sigma^*[o_T(x) = q_A \Leftrightarrow x \in L^c]$.

Costruisco una macchina di turing T' con input x.
1) Simula 1 sola istruzione di $T(x)$, tale che:

- se $o_T(x) = q_A$ allora accetta
- altrimenti va in 2

2) Simula 1 sola istruzione di $T^c(x)$, tale che:

- se $o_{T^c}(x) = q_A$ allora rigetta
- altrimenti va in 1

12 Funzioni Calcolabili e Linguaggi Decidibili

Un linguaggio L è decidibile se esiste una macchina di Turing deterministica T tale che, per ogni input x:

- T accetta se $x \in L$,
- T rifiuta se $x \notin L$,
- T termina sempre.

Dato un linguaggio $L \subseteq \Sigma^*$, la sua funzione caratteristica $\chi_L : \Sigma^* \to \{0,1\}$ è la funzione totale definita come:

$$\chi_L(x) = \begin{cases} 1 & \text{se } x \in L, \\ 0 & \text{se } x \notin L. \end{cases}$$

Teorema: Un linguaggio L è decidibile se e solo se la sua funzione caratteristica χ_L è calcolabile.

DIM:

($\Rightarrow$) Supponiamo che L sia decidibile.
Allora esiste una macchina di Turing deterministica T di tipo riconoscitore tale che, per ogni $x \in \Sigma^*$:

$$T(x) = q_A \iff x \in L, \qquad T(x) = q_R \iff x \notin L.$$

In particolare, poiché L è decidibile, T termina sempre su ogni input.

Costruiamo ora una macchina di Turing T' di tipo trasduttore, a due nastri (1 di input, 1 di output), che calcola la funzione caratteristica χ_L.
Su input $x \in \Sigma^*$ la macchina opera come segue:

1. Sul primo nastro simula la computazione di $T(x)$.
2. Sul secondo nastro, scrive il risultato. Quindi se $T(x)$ termina nello stato q_A, allora T' scrive 1 sul nastro di output; altrimenti scrive 0 e termina.

Poiché T termina sempre, anche T' termina sempre.
Se $x \in L$, allora $T(x) = q_A$ e quindi T' scrive 1;
se $x \notin L$ allora $T(x) = q_R$ e T' scrive 0.
Pertanto χ_L è calcolabile.

($\Leftarrow$) Supponiamo ora che χ_L sia calcolabile.
Allora esiste una macchina di Turing T di tipo trasduttore tale che, per ogni $x \in \Sigma^*$, T termina e scrive sul nastro di output il valore $\chi_L(x)$, cioè:

$$\chi_L(x) = \begin{cases} 1 & \text{se } x \in L, \\ 0 & \text{se } x \notin L. \end{cases}$$

Vogliamo costruire una macchina di Turing T' che decida il linguaggio L.
Definiamo T' come una macchina riconoscitore a due nastri che opera nel modo seguente:

1. Sul primo nastro simula $T(x)$, scrivendo il suo output sul secondo nastro
2. Se sul secondo nastro c'è scritto 1, allora $T'(x)$ entra nello stato di accettazione q_A; se c'è scritto 0, entra nello stato di rifiuto q_R.

Poiché χ_L è calcolabile, la simulazione di T al passo (1) termina sempre.
Se $\chi_L(x) = 1$, allora T' accetta; se $\chi_L(x) = 0$, allora T' rifiuta.
Quindi T' termina sempre su ogni input e decide il linguaggio L.
Concludiamo dunque che L è decidibile se e solo se la funzione caratteristica χ_L è calcolabile.

Teorema: Se la funzione $f : \Sigma^* \to \Sigma_1^*$ è totale e calcolabile allora il linguaggio $L_f \subseteq \Sigma^* \times \Sigma_1^*$ (linguaggio che contiene tutte le coppie) è decidibile.

N.B: In altre parole, stiamo affermando che, se possiamo sempre calcolare la funzione f, allora possiamo sempre decidere se una data coppia input-output (x, y) è corretta.

Poichè f è calcolabile e totale, allora esiste una Macchina di Turing di tipo trasduttore che calcola $f(x)$, per ogni $x \in \Sigma^*$. Definiamo una macchina di Turing di tipo riconoscitore T' a 2 nastri, con input $< x, y >$, dove $x \in \Sigma^* s$ e $y \in \Sigma_1^*$. Avremo:

- Nel primo nastro l'input $< x, y >$,
- Nel secondo nastro, dopo aver eseguito la computazione $T(x)$, scrive z (ovvero $z = f(x)$). Successivamente confronta z con y. Infine la computazione termina nello stato di accettazione q_A, altrimenti termina nello stato di rigetto q_R.

Questo ci mostra che la Macchina di Turing T' che abbiamo costruito è un decisore per il linguaggio L_f.
Essere un decisore significa che T' deve sempre terminare (o nello stato di accettazione o in quello di rigetto) e deve dare la risposta corretta (accetta solo le coppie $\langle$x,y$\rangle$ per cui $f(x) = y$.

Teorema: Sia $f : \Sigma^* \to \Sigma_1^*$ una funzione. Se il linguaggio $L_f \subseteq \Sigma^* \times \Sigma_1^*$ è decidibile allora f è calcolabile.

DIM: Per ipotesi, il linguaggio $L_f = \{\langle x, y\rangle \mid y = f(x)\}$ è decidibile. Questo implica che esiste una Macchina di Turing T di tipo riconoscitore (ovvero una macchina che decidere se l'ìnput appartiene d un insieme) che, per ogni coppia $\langle x, y\rangle$ con $x \in \Sigma^*$ e $y \in \Sigma_1^*$, garantisce di terminare nello stato:

$$T(\langle x, y\rangle) = \begin{cases} q_A & \text{se } y = f(x) \\ q_R & \text{altrimenti} \end{cases}$$

(dove q_A è lo stato di accettazione e q_R è lo stato di rigetto).
Poiché L_f è decidibile, T termina sempre.

Definiamo una nuova Macchina di Turing T' di tipo trasduttore (ovvero che calcola un risultato) con $a \geq 1$ nastri. Questa macchina prenderà in input $x \in \Sigma^*$ e deve produrre in output $y = f(x)$. Tutto ciò funziona nel seguente modo:

- T' riceve x sul primo nastro.
- T' enumera tutte le possibili stringhe $y \in \Sigma_1^*$ in ordine lessicografico per lunghezza crescente: prima le stringhe di lunghezza 0, poi quelle di lunghezza 1, e così via. Questo processo garantisce che ogni stringa y di lunghezza finita venga considerata.
- Per ogni stringa y generata, T' simula la computazione del Verificatore T sull'input $\langle x, y\rangle$.

  $$T' \text{ simula } T(\langle x, y\rangle)$$

- Poiché T è un decisore, la simulazione al passo 3 termina sempre in un numero finito di passi.
  - Se T termina in q_A (accettazione), allora $y = f(x)$. T' scrive y sul nastro di output e termina.
  - Se T termina in q_R (rigetto), allora $y \neq f(x)$. T' scarta y e torna al passo 2 per generare la prossima stringa.

Dobbiamo dimostrare che, se x è nel dominio di f, T' termina con l'output corretto. Se x appartiene al dominio di f, allora esiste una stringa univoca $y_0 \in \Sigma_1^*$ tale che $y_0 = f(x)$. Poiché T' enumera sistematicamente tutte le stringhe di Σ_1^*, prima o poi T' genererà la stringa y_0. Quando T' simula $T(\langle x, y_0\rangle)$, la computazione T termina in q_A (per definizione di L_f). Di conseguenza, T' scriverà y_0 sul nastro di output e terminerà. Dunque, abbiamo costruito una MdT T' che calcola la funzione f. Per definizione, la funzione f è **calcolabile**. Se x non appartiene al dominio di f, non esiste nessuna stringa y che causerà l'accettazione da parte di T. In tal caso, T' continuerà all'infinito a generare stringhe e a rigettarle, e quindi T' **non terminerebbe**. Questo è coerente con

la definizione di funzione calcolabile, che richiede l'arresto solo per gli input nel dominio della funzione.

13 La Tesi di Church-Turing

Nella Macchina di Turing nella teoria della calcolabilità è stata un'ampia varietà di modelli di calcolo.
Per tutti questi modelli di calcolo, è stato dimostrato che essi sono Turing-equivalenti, ossia, qualunque funzione calcolabile mediante uno di tali modelli è calcolabile anche mediante una macchina di Turing.
Questo ha portato due logici: Church e Turing ad enunciare una tesi.

È calcolabile tutto (e solo) ciò che può essere calcolato da una macchina di Turing.

Teorema: Per ogni programma scritto in accordo con il linguaggio di programmazione PascalMinimo, esiste un macchina di Turing T di tipo trasduttore che scrive sul nastro di output lo stesso valore fornito in output dal programma.
Dimostrazione omessa

14 Quanti sono gli Algoritmi?

Qual'è la cardinalità degli insieme degli algoritmi?
La tesi di Church - Turing afferma che:
Ad ogni algoritmo corrisponde una macchina di Turing che calcola la stessa funzione.
Per ogni macchina di Turing definita su un generico alfabeto e dotata di k nastri esisterà un'altra macchina di Turing definita sull'alfabeto $\{0, 1\}$ e dotata di un singolo nastro.

Teorema: L'insieme delle macchine di Turing definite sull'alfabeto $\{0, 1\}$ e dotate di un singolo nastro è numerabile.

DIM: Per dimostrare che un insieme è numerabile, dobbiamo dimostrare che esiste un modo per assegnare a ogni elemento dell'insieme delle macchine di Turing un'etichetta numerica unica, ovvero un $n \in \mathbb{N}$.

Sia T una macchina di Turing ad un Nastro, definita sull'alfabeto $\{0, 1\}$. Questa solitamente è definita da una Funzione di Transizione. Questa può essere vista come un elenco di istruzioni, chiamate Quintuple.
Dato che la macchina ha un iniseme finito di stati e usa un alfabeto finito, l'intera descrizione della macchina è un elenco finito di queste quintuple.

Possiamo trasformare queste istruzioni in una stringa continua β_T e la scriveremo nel seguente modo:

$$\beta_T = b(q_i) \otimes a - b(q_j) \otimes b \otimes m$$

Dove:

- $b(q_n)$ è la rappresentazione binaria dello stato.
- $\otimes$ funge da separatore tra gli elementi della stessa regola.
- $-$ separa la condizione (input) dall'azione (output).
- $m \in \{f, s, d\}$ indica il movimento della testina (fermo, sinistra, destra).
- $\oplus$ è il separatore usato per unire diverse istruzioni in una sola sequenza.

Una volta ottenuta la stringa β_T, applichiamo una funzione di sostituzione per eliminare i simboli non numerici e ottenere una stringa composta solo da cifre:

Simbolo	Cifra Associata
s	5
f	6
d	7
$-$	4
$\otimes$	3
$\oplus$	2

Dato che una codifica potrebbe iniziare con la cifra 0 si premette la cifra 8 alla sequenza ottenuta, poichè se trattata come un numero intero questo causerebbe la perdita di informazione.
Poichè ogni macchina, adesso è un numero, puoi metterle tutte in fila e contarle. Se puoi contarle, l'insieme è numerabile.

15 Halting Problem

Consideriamo la funzione $v : \mathscr{T} \to \mathbb{N}$ definiamo il linguaggio seguente:

$$L_H = \{(i,x) : i \text{ contiene solo le cifre da '0' a '7'} \wedge i \text{ è la codifica di una macchina di Turing} \wedge T_i(x) \text{ termina } \mathbb{N} \times \mathbb{N}\}$$

Teorema:
L_H è un linguaggio accettabile, ma non decidibile.

DIM (se è accettabile): Dobbiamo dimostrare che esiste una macchina di Turing T tale che, per ogni input $(i,x) \in \mathbb{N} \times \mathbb{N}$,

$$T(i,x) = q_A \text{ se e soltanto se } (i,x) \in L_H.$$

Definiamo U' una macchina di Turing universale modificata con input (i,x). Tale macchina opera nel seguente modo:

1. Verifica se i è la codifica di una macchina di Turing. Se non lo è allora $U'(i,x) = q_R$, va nello stato di rigetto.
2. Simula $U(i,x)$, se termina in q_A o in q_R allora $U'(i,x) = q_A$.

U' non sa decidere L_H^c, perciò lo accetta solo.

DIM: Supponiamo che L_H sia decidibile.

$$\text{Ovvero } L_H \text{ è decidibile } \iff L_H \text{ e } L_H^c \text{ sono accettabili.}$$

Allora, se L_H è decidibile deve esistere una macchina di Turing T tale che:

$$T(i,x) = \begin{cases} q_A & \iff (i,x) \in L_H \\ q_R & \iff (i,x) \notin L_H \end{cases}$$

1. Derivazione di $T' \to$ Avendo assunto prima che L_H sia decidibile tramite una macchina T, possiamo derivare una macchina T' che si comporta in modo opposto.
Formalmente, definiamo T' scambiando gli stati di T in modo che:

$$T'(i,x) = \begin{cases} q_A & \iff (i,x) \notin L_H \\ q_R & \iff (i,x) \in L_H \end{cases}$$

2. Derivazione di T'' $\rightarrow$ Trasformiamo il rigetto della macchina T' in un ciclo infinito. Ovvero se la macchina T' entrava nello stato di rigetto q_R, questa macchina T'' non terminerà ma entrerà in un ciclo infinito.

$$T''(i,x) = \begin{cases} q_A & \iff (i,x) \notin L_H \\ \text{non termina} & \text{se } (i,x) \in L_H \end{cases}$$

3. Derivazione di T^* $\rightarrow$ Se diciamo che T esiste, allora anche T^* esiste (perché è solo una versione modificata di T).
Se T esiste, deve far parte dell'insieme di tutte le Macchine di Turing possibili. Siccome ogni macchina ha un numero d'indice codificato (chiamiamolo k), allora deve esistere un k tale che $T_k = T$.
Diamo alla macchina in input, il suo stesso numero d'indice k. Qui genereremo il paradosso:

$$T^*(i) = T''(i,i) = \begin{cases} q_A & \iff (i,i) \notin L_H \\ \text{non termina} & \text{se } (i,i) \in L_H \end{cases}$$

Conclusione $\rightarrow$ Sia k l'indice della macchina T^*, ovvero $T^* = T_k$. Dalla definizione di T^* otteniamo:

$$T^*(k) \text{ termina} \iff (k,k) \notin L_H \iff T_k(k) \text{ non termina}$$

Poiché $T^* = T_k$, abbiamo la contraddizione finale:

$$T_k(k) \text{ termina} \iff T_k(k) \text{ non termina}$$

Questo assurdo logico dimostra che l'ipotesi iniziale è falsa: L_H non è decidibile.

N.B: Teorema 5.6 svolto come esercizio.

16 Altri Teoremi:

Se L_1 e L_2 sono due linguaggi accettabili, allora L_1L_2 è un linguaggio accettabile. Se L_1 e L_2 sono due linguaggi decidibili, allora L_1L_2 è un linguaggio decidibile.
DIM sta nel quaderno

Se L_1 e L_2 sono due linguaggi accettabili, allora L_1L_2 è un linguaggio accettabile. Se L_1 e L_2 sono due linguaggi decidibili, allora L_1L_2 è un linguaggio decidibile.
DIM sta nel quaderno

Riduzioni fra Linguaggi (many to one)

Sia $L_1 \subseteq \Sigma_1^*$ un linguaggio e sia $L_2 \subseteq \Sigma_2^*$.
Definisco L_1 riducibile a L_2 (che indicheremo con il seguente simbolo $L_1 \leq L_2$) se esiste una funzione $f : \Sigma_1^* \longrightarrow \Sigma_2^*$ tale che per ogni input $x \in \Sigma_1^*$

$$x \in L_1 \Leftrightarrow f(x) \in L_2$$

[esiste un trasduttore che calcola una parola $y = f(x)$ per cui vale
$x \in L_1 \Leftrightarrow y \in L_2$]

Teorema 1: Se $L_1 \leq L_2$ e L_2 è un linguaggio decidibile. Allora L_1 è decidibile

Teorema 2: Se $L_1 \leq L_2$ e L_2 è un linguaggio accettabile. Allora L_1 è accettabile

Teorema 3: Se $L_1 \leq L_1$ e L_1 è un linguaggio NON decidibile. Allora L_2 non è decidibile
DIM: Dimostriamo per assurdo
Suppongo che L_2 è decidibile.
Allora poichè per ipotesi $L_1 \leq L_2$ anche L_1 sarebbe decidibile contraddicendo l'ipotesi che L_1 non è decidibile